Final Project for SW Engineering Class CSC 648-848 Spring 2026

GatorBazaar

Team 07

Members	
Team Lead	Dylan Cordoni (dcordoni@sfsu.edu)
Github Maintainer/Front-End	Abhay Bajaj
Back-End Lead	Sebastian Cervantez
	Ryan Zhang
Front-End Lead	Daniel De Jesus
	Kenny Camarra
	Diego Rivera-Soto

Milestone 5

Url: http://3.150.110.210

Date: 05/20/2026

Table of Contents

Table of Contents....2
1. Product Summary....3
2. Milestone Documents....5
Milestone 1....5
Milestone 2 Part I....20
Table of Contents....21
1. Executive Summary:....22
2. Main data items and entities:....23
3. Functional Requirements....25
4. UI Storyboards....26
5. High Level Architecture/Database Organization:....38
6. Key Risks....41
7. Project Management....42
8. Use of GenAI Tools:....43
9. Team Lead Check List:....47
M3 Feedback Summary From In-Person Review Meeting:....48
Milestone 4....52
1. Product Summary....53
2. Usability test plan for selected function....54
3. QA test plan and QA testing....56
4. Peer Code Review:....58
5. Self-check on best practices for security – ½ page....68
6. Self-check of the adherence to original Non-functional specs....70
3. Product Screenshots....72
4. Database Organization:....83
5. GitHub organization:....85
6. Google analytics stats plot for your WWW site....87
7. Project management:....88
8. Team member self-assessment and contributions....89

1. Product Summary

Product name: GatorBazaar

Gator Bazaar is a web marketplace built for San Francisco State University students to buy, sell, and trade campus essentials from textbooks and electronics to furniture and room listings safely with other Gators.

Major functions (P1 — final commitment)

Each item below is a capability your team will deliver and test for grading:

• Students sign-up using an @sfsu.edu email address so only SFSU students can create accounts.
• Sign-in with username and password; the site remembers who you are while you browse.
• Browse the home page by student-focused categories (e.g., clothes, textbooks, furniture, electronics, school supplies, room search).
• Search the marketplace by keywords from the main search bar.
• Refine search results with category filters, a price range, and sort options e.g., lowest/highest price, newest.
• Open a listing to see item details: title, price, description, category, image(s), and seller information.
• Save listings to favorites when signed in, and remove them from favorites.
• View a favorites page that shows all items you have saved.
• Message a seller from a listing page when signed in guests are prompted to register or sign in.
• Use an inbox to view conversations and send replies between buyers and sellers about specific listings.
• Use a seller dashboard to see your active listings and listings you have marked as sold.
• View your account page with profile details (e.g., name, username, join date) when signed in.
• Create and post new listings for sale with title, price, description, category, and image (seller flow).
• Complete a transaction on a transaction summary page: review details, confirm the exchange, and submit ratings/feedback (buyer and seller views).

What is unique and SFSU-specific?

• Built for San Francisco State University students only registration requires an @sfsu.edu email.
• Campus-oriented categories which include textbooks, school supplies, and room search, not generic national marketplace categories.
• Peer-to-peer on and near campus listings and flows are aimed at students meeting and trading locally (e.g., campus-area meetup context in transaction content).
• Course project context branded as an SFSU CSC 648/848 Team 07 application for the Spring 2026 software engineering project.

URL: http://3.150.110.210

2. Milestone Documents

SW Engineering CSC648-848 Spring 2026

GatorBazaar

Team 07

Members	
Team Lead	Dylan Cordoni (dcordoni@sfsu.edu)
Github Maintainer	Abhay Bajaj
Back-End Lead	Sebastian Cervantez
	Ryan Zhang
Front-End Lead	Daniel De Jesus
	Kenny Camarra
	Diego Rivera-Soto

Milestone 1

History Table	
Date Submitted	March 18, 2026
Date Revised	March 30, 2026

1. Executive Summary:

College students often have so much stuff towards the end of their four years and even semesters. These items include textbooks from past semesters, small furniture, electronics, or school supplies. At the same time, many other students are looking for affordable alternatives instead of buying new items. However, existing marketplaces such as general online classifieds are not designed specifically for university communities and can feel inconvenient or unsafe for student use. Having to meet up in locations that aren't always safe, for example Facebook marketplace, etc. GatorBazaar is a marketplace platform designed specifically for the San Francisco State University (SFSU) community where students and professors can easily buy, sell, and exchange items within a trusted campus environment.

GatorBazaar provides a simple web-based platform where users can list items they no longer need and browse items posted by others. Listings are organized into categories such as textbooks, electronics, clothing, and dorm essentials, and many other features we have coming down the pipeline, making it easy for users to quickly find what they are looking for. Students can post items for sale, upload descriptions and prices, and connect with buyers within the SFSU community. Professors can also use the platform to share or sell academic materials or recommend resources to students. By centralizing these exchanges in one place, GatorBazaar helps reduce waste, saves students money, and creates a convenient and safe way to reuse items within the campus community.

As San Francisco State University students ourselves, our team is dedicated to making a marketplace that not only our fellow students feel confident and safe using, but one that we would use as well. With student safety and convenience at our highest priority, we believe we can make GatorBazaar the ideal cornerstone for second-hand student shopping.

2. Personae:

Persona 1:

Name: John

Year: Incoming Sophomore

John, who just finished his first year of college, plans to move out of his school campus dorm and settle in an apartment near his college. He works a part-time job to pay for rent, but still receives some help from his parents, and he plans to find a roommate or two to cut the cost of rent he has to pay. His goal is to find affordable furniture near campus to decorate his apartment without spending too much and to avoid unsafe spaces. He's proficient in using web applications and knows how to navigate, but he's not experienced with buying secondhand items. He wants to buy furniture from another student near campus so he feels safe and since furniture can be expensive, buying secondhand items with the help of our application helps him find cheap options.

Persona 2:

Name: Jose Corona

Year: Freshman

Jose Corona is a new student at San Francisco State University. He is originally from San Diego, California and moved to SF to study Business Administration at SF State. As a new student at SF State, Jose lives in the dorms at SF State and is not familiar with the campus, city, and community around him. Like many students, his professors this semester have assigned him textbooks to buy for the semester. Although he has been told that he can try to download the books online, Jose prefers to buy the books and wants to find the best deals possible. Jose needs the books as soon as possible, and he doesn't want to buy the books online and believes he can find a better deal than the prices at the SF State book store. Therefore, Jose wants to browse the GatorBazaar app to look for the textbooks he needs for his classes.

Persona 3:

Name: Javier Hernandez

Year: Senior

Javier is a senior at San Francisco State studying biology. He is from Los Angeles and lives in a shared apartment near SF State. As a student who moves back to his home in Los Angeles after the semester ends, Javier has to get rid of the things he cannot carry on his flight back home. Therefore, Javier decides to use GatorBazaar to sell his items that he no longer wants. These items include items such as textbooks, clothes, electronics, appliances, class supplies and other things that are in his room he no longer wants. Javier is a very flexible man so he accepts cash or Zelle payments and has relatively low prices since his overall goal is to get rid of the items he no longer wants.

Persona 4:

Name: Khalid Kashmiri

Year: Senior

Khalid is a senior student at SFSU and is one of the admins for the GatorBazaar. He is a business student and is responsible for moderating the marketplace in order to make sure the market is safe and organized. Khalid's job is to review user reports and prevent scams. He is focused on maintaining a safe and positive market for his fellow students and has had training and previous experience in moderating people and websites in other online communities. Some of his biggest issues he encounters are spam listings and posts that violate the platform's rules, reducing others' trust in GatorBazaar. Khalid wants to make sure that the platform stays safe, positive, and has a high trust among students at SFSU.

Persona 5:

Name: Baljeet Singh

Year: Incoming Freshman

Baljeet is an International Student from India, majoring in Computer Science. Coming to a new country and getting used to the new things here is a bit challenging for him. He is planning to dorm and make some new friends. He wants to make sure he has everything ready before classes start. He moves in a few days before classes start, so he wants to purchase things that will help him get through his first semester. He wants to look into a study desk, so he can be locked in, but they all happen to be very expensive in stores and online. So now, he looks into the GatorBazaar app, trying to find himself a nice desk for the best price and also look for the required textbooks for his class.

3. High-level Use cases:

Use Case 1: Buying Furniture Using GatorBazaar

John has just completed his first year at SFSU and is moving from his dorm into an apartment near campus. Because he's working part-time and wants to minimize his expenses, he looks for affordable secondhand furniture. He uses GatorBazaar, which is limited to verified SFSU students through their university login. This gives him safety knowing that he would only be meeting up with other students and not strangers. He filters listings to show items he needs like tables, bed frames, chairs, and shelves so everything he needs is organized. Since John is not experienced with buying secondhand furniture, knowing the seller is an SFSU student and meeting in campus locations or nearby makes him feel safer. The application allows him to furnish his apartment affordably while maintaining a sense of security within the SFSU community.

Use Case 2: Buying Textbooks Using GatorBazaar

Jose's classes require him to get those textbooks, so he decides to use GatorBazaar to search for the books at a good price. By searching the listing tags for the courses he's taking, he's able to easily find listings specific to those courses. This way, not only can he find the required textbooks more easily, but he can make sure that they're the ones he needs for his courses. Additionally, since the app can only be used by SF State students and faculty, he can assure that the materials are those used for SF State classes and that the person selling them can authenticate the use of the textbook in the class. Also, since Jose is unfamiliar with San Francisco, he prefers to use the app since he can choose to meet up at a location on campus that will facilitate purchasing items.

Use Case 3: Selling Items Using GatorBazaar

Javier has to move out of his apartment by the end of the semester he uses Gator Bazaar to sell the stuff in his apartment that he currently does not want. Since the app is only for SF State students and faculty he knows that he can target and sell to students who might want to buy his items. Javier can log into the app and post the items he wants to sell such as a chair and desk and list them under furniture so that other students can buy his items.

Use Case 4: Finding a Roommate Through GatorBazaar

Jose is finishing his first year of college and plans to move out of the dorms the following semester, he wants to find a roommate to help reduce the cost of rent. He doesn't want to live too far out of campus and with non-students, so he uses GatorBazaar to connect with other verified SFSU students who are also looking for housing. By using the app, he can search for listings from students who are looking for roommates in nearby apartments. Since the platform is exclusive to SF students, Jose feels more comfortable communicating and arranging meetings on or near campus.

Use Case 5: Moderating the Platform with Admin Tools.

Khalid is spending some time reviewing listings posted by users on GatorBazaar in order to ensure that they follow the platform's rules and guidelines in order to keep SFSU students safe and their trust in the platform high. When Khalid finds a listing that violates the rules, or sees a listing that was reported he will remove the listing that is active, and if the SFSU user who posted the listing has a low rating score, Khalid may suspend that reported users account based on previous violations.

4. List of main data items and entities – data glossary/description:

- **Admin Users** - Users that manage reports received by either listings or users. Able to issue warnings to users about violations or take action in the form of removing a listing or banning a registered user. Reports will be displayed on the Registered Users profile or Listing Page.
- **Unregistered Users** - Users who can only access the homepage, browse, search, and filter listings, and view listing details. Do not need to log in or register.
- **Registered Users** - Users that have signed up on the site using a SFSU verified email.
    - **Buyers** - Registered users looking to purchase listings on the site. All registered users are treated as buyers innately.
        - **Sent Request** - Used to show a buyer's interest in a listing that is sent to the seller.
        - **Sent Request List** - A list showing all items a buyer has sent a request to buy (seen by buyer only).
        - **Bought Listings List** - A list of listings previously purchased by a buyer.
    - **Sellers** - Users that have at least one listing on the site.
        - **Listing -** An item for sale posted by and belonging to a registered user.
            - **Title -** The name of the item for sale.
            - **Image** - An image of the item for sale set by the seller.
            - **Description -** A description of the item for sale set by the seller.
            - **Price** - Price of the item set by the user.
            - **Category** - Grouping of a listing provided by the site.
            - **Tags** - Additional searchable terms set by the seller.
                - **Course** - Sellers can also set an SFSU course as a tag for listings related to a course.
            - **Pending Request** - A request received by the seller used to show a buyer's interest in the listing.
            - **Listing Report** - A report specific to a listing sent by other users if the listing breaks the guidelines of the site (seen by admin)
            - **Active -** Shows whether listing is currently being sold or not (system purposes)
        - **Active Listing List** - A list of all listings a seller currently has on the user's page (seen by all users)
        - **Sold Listing List** - A list of all past listings sold by a seller on the user's page (seen by user owning the list and admins)
    - **Messaging** - Allows communication between the seller and the buyer.
        - **Message** - A communication instance between registered users.

        - **Inbox** - A list of current and past messages between registered users. Allows registered users to send messages to other registered users.
        - **Notification** - When a registered user receives a message from another registered user, their inbox will indicate that they have a new message.
    - **Meet up** - A tracked location on campus where the buyer and seller agree to enact the exchange.
    - **User Rating** - Aggregate rating given to a seller/buyer by other users (seen by all users)
    - **User Report** - Report specific to a registered user sent by other registered users if guidelines of the site are broken by the reported user. (seen by admins)
    - **Email Verification** - Checks whether a user attempting to register has a valid SFSU email.
- **Search** - A method of filtering listings by their attributes(name, tags, price, etc.)
    - **Roommate Search** - Searching listings within the apartment category to find roommates.

5. Functional Requirements

Unregistered Users (5 Reqs)

1. The system shall allow visitors to access the marketplace homepage.
2. The system shall allow visitors to browse publicly available item listings.
3. The system shall allow visitors to search listings using keywords.
4. The system shall allow visitors to filter listings by category.
5. The system shall allow visitors to view listing details including title, description, price, and images.

Registered Users (11 Reqs)

6. The system shall allow users to register using an SFSU email address.
7. The system shall verify that registered users are associated with an SFSU email domain.
8. The system shall allow users to receive notifications when they receive messages.
9. The system shall allow users to report listings that violate platform rules.
10. The system shall allow users to give a rating to either seller or buyer.

Seller Specific:

11. The system shall allow registered users to create listings for items they wish to sell.
12. The system shall allow users to upload images when creating listings.
13. The system shall allow users to list a filter to their listing such as course-specific listings, furnitures, or rooms.
14. The system shall allow users to edit their listings.

15. The system shall allow users to delete their listings.
16. The system shall allow users to mark listings as sold.

Buyer Specific:

17. The system shall allow users to initiate the sending of messages to other users regarding listings.
18. The system shall allow users to search with word-specific filters to browse for listings.

Administrative Users (5 Reqs)

19. The system shall allow administrators to review reported listings.
20. The system shall allow administrators to remove listings that violate policies.
21. The system shall allow administrators to suspend users who repeatedly violate rules.
22. The system shall allow administrators to monitor marketplace activity.
23. The system shall allow administrators to manage listing categories.

6. Non-Functional Requirements

1. Application shall be developed, tested and deployed using tools and cloud servers approved by Class CTO and as agreed in M0
2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers
3. All our selected application functions shall be rendered well on mobile devices (no native app to be developed)
4. Posting sales information and messaging to sellers shall be limited only to SFSU students
5. Critical data shall be stored in the database on the team's deployment server.
6. No more than 50 concurrent users shall be accessing the application at any time
7. Privacy of users shall be protected
8. The language used shall be English (no localization needed)
9. Application shall be very easy to use and intuitive
10. Application shall follow established architecture patterns
11. Application code and its repository shall be easy to inspect and maintain
12. Google analytics shall be used

13. No e-mail clients or chat services shall be allowed. Interested users can only message to sellers via in-site messaging. One round of messaging (from user to seller) is enough for this application
14. Pay functionality, if any (e.g. paying for goods and services) shall not be implemented nor simulated in UI.
15. Site security: basic best practices shall be applied (as covered in the class) for main data items
16. Media formats shall be standard as used in the market today
17. Modern SE processes and tools shall be used as specified in the class, including collaborative and continuous SW development and GenAI tools
18. The application UI (WWW and mobile) shall prominently display the following exact text on all pages "SFSU Software Engineering Project CSC 648-848, Spring 2026. For Demonstration Only" at the top of the WWW page Nav bar. (Important to not confuse this with a real application). You must use this exact text without any editing.

7. Competitive Analysis:

Feature	Facebook Marketplace	Craigslist	OfferUp	Mercari	(Our App) GatorBazaar
Verified/Secure Identity Accounts	X	X	✓	✓	✓✓✓
Message between Buyer & Seller	✓✓✓	✓	✓	✓✓✓	✓
Supports In-person Meetups/Exchanges	✓	✓	✓	X	✓✓✓
Campus-specific Users & Features	X	X	X	X	✓✓✓

Browse & Search Listings	✓✓✓	✓✓✓	✓✓✓	✓✓✓	✓
In-app Purchases/Payment	**X**	**X**	✓	✓	**X**
Roommate finding	✓	✓	**X**	**X**	✓✓✓

X = Does not support function, ✓ = Does support function, ✓✓✓ Does support and is superior

Existing online marketplaces such as Facebook Marketplace, Craigslist, OfferUp, and Mercari allow users to buy and sell items locally. These platforms provide common marketplace features such as browsing listings, messaging between buyers and sellers, and posting items for sale. However, these platforms are designed for the general public, meaning users often interact with strangers and may need to travel significant distances to complete exchanges. Our proposed SFSU Marketplace, GatorBazaar, focuses specifically on the SFSU student community market. By requiring verification through an SFSU email address, the platform creates a more trusted environment where users know they are interacting with fellow SFSU students and faculty. In addition, the system prioritizes convenient on-campus meetups rather than shipping or online payments, making exchanges simpler and safer for students. This focus on a verified campus community provides a targeted advantage over existing public marketplace platforms for common products that users may want to exchange in a close, local environment.

8. High-level system architecture and technologies used:

Software stack:

- Server Host: AWS EC2 t3.small
- Operating System: Ubuntu 24.04.1
- Database: MySQL v 8.0.45
- Web Server: Apache 2.4.58
- Server-Side Language: JavaScript

Additional Technologies:

- Web Framework:
  - Express.js v 5.2.1
  - Node.js v 22.12.0
- Web Analytics: Google Analytics

SSL Cert: Lets Encrypt (Cert Bot)
React: 19.2.0

Browsers Supported:

Google Chrome version 142 and later
FireFox version 146 and later

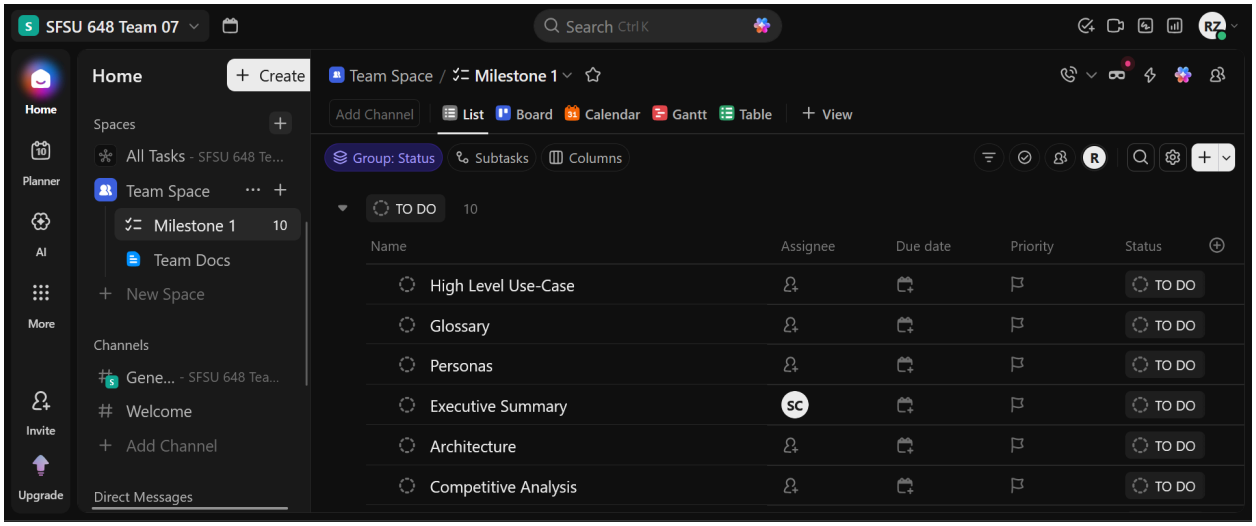
9. Use of GenAI tools like ChatGPT and Copilot for Milestone 1:

We used ChatGpt 5.2 in the process of working on M1 for tasks like:

- Deciding on a project name: Medium
    - We used the prompt “come up with a name for my website which is a marketplace for SFSU students and faculty” to get suggestions for our project name, since we had difficulty coming up with our own. From these suggestions we voted on a specific name to use. While it would’ve been easy to come up with our own names, we liked the names it suggested and it helped us reduce this process to only a few minutes.
- Summarize/condense a draft for the Competitive Analysis: High
    - We used the following prompt, “Summarize in one paragraph what are the planned advantages or competitive relationship of your planned product to what is already available. In the table clearly mark your product, e.g. shade its column/data. Your SFSU specific functions must also be addressed in competitive analysis and shown in the list of compared functionalities. I already have a table, and a competitive analysis, but I wrote it too long and I think it needs some revision to fit with the requirements listed above. Below is my table and summary.” for this task. The output for the prompt was good, and was used as a basis of our final Competitive Analysis summary, but still needed some tweaks to the grammar and to fit our purposes better.

10. Task Management Tool Selection:

Task Management Tool: Click Up



11. Team and Roles:

Roles Table		
Name	Email	Role
Dylan Cordoni	dcordoni@sfsu.edu	Team Lead
Daniel De Jesus	jdejesus1@sfsu.edu	Front-End Lead

Roles Table		
Ryan Zhang	rzhang13@sfsu.edu	Back-End
Kenny Camarra	kcamarra@sfsu.edu	Front-End
Abhay Bajaj	abajaj@sfsu.edu	Github Maintainer/Front-End
Diego Rivera-Soto	driverasoto@sfsu.edu	Front-End
Sebastian Cervantez	scervantez@mail.sfsu.edu	Back-End Lead

12. Team Lead Check List:

Check List	
Done	All team members are fully engaged and attend team sessions when required
Done	Team found a time slot to meet outside of class
Done	Team ready and able to use the chosen back and front-end framework and those who need to learn are working on learning and practicing
Done	Team reviewed class slides on requirements and use-cases before drafting Milestone 1
Done	Team leads checked Milestone 1 document to quality, completeness, formatting and compliance with instructions before the submissions
Done	Team shared and discussed experience with GenAI tools among themselves
Done	GitHub organized as discussed in class (e.g. master branch, development branch, folder for milestone document etc.
Done	Team has chosen task management tool and invited Course instructor to project via ajsouza@sfsu.edu email
Done	Team reviewed non-functional requirements from the How to Start document and developed Milestone 1 consistently.

Check List	
Done	Team lead ensured that all team members read the final M1 and agree and understand it before submission.

SW Engineering CSC648-848 Spring 2026

GatorBazaar

Team 07

Members	
Team Lead	Dylan Cordoni (dcordoni@sfsu.edu)
Github Maintainer	Abhay Bajaj
Back-End Lead	Sebastian Cervantez
	Ryan Zhang
Front-End Lead	Daniel De Jesus
	Kenny Camarra
	Diego Rivera-Soto

Milestone 2 Part I

History Table	
Date Submitted	April 05, 2026
Date Revised	April 18, 2026

Table of Contents

Table of Contents.....2

1. Executive Summary:..... 3

2. Main data items and entities:.....4

3. Functional Requirements.....6

4. UI Storyboards.....7

5. High Level Architecture/Database Organization:..... 19

6. Key Risks.....22

7. Project Management.....23

8. Use of GenAI Tools:.....24

9. Team Lead Check List:.....27

1. Executive Summary:

College students often have so much stuff towards the end of their four years and even semesters. These items include textbooks from past semesters, small furniture, electronics, or school supplies. At the same time, many other students are looking for affordable alternatives instead of buying new items. However, existing marketplaces such as general online classifieds are not designed specifically for university communities and can feel inconvenient or unsafe for student use. Having to meet up in locations that aren't always safe, for example Facebook marketplace, etc. GatorBazaar is a marketplace platform designed specifically for the San Francisco State University (SFSU) community where students and professors can easily buy, sell, and exchange items within a trusted campus environment.

GatorBazaar provides a simple web-based platform where users can list items they no longer need and browse items posted by others. Listings are organized into categories such as textbooks, electronics, clothing, and dorm essentials, and many other features we have coming down the pipeline, making it easy for users to quickly find what they are looking for. Students can post items for sale, upload descriptions and prices, and connect with buyers within the SFSU community. Professors can also use the platform to share or sell academic materials or recommend resources to students. By centralizing these exchanges in one place, GatorBazaar helps reduce waste, saves students money, and creates a convenient and safe way to reuse items within the campus community.

As San Francisco State University students ourselves, our team is dedicated to making a marketplace that not only our fellow students feel confident and safe using, but one that we would use as well. With student safety and convenience at our highest priority, we believe we can make GatorBazaar the ideal cornerstone for second-hand student shopping.

2. Main data items and entities:

- **Admin Users** - Users that manage reports received by either listings or users. Able to issue warnings to users about violations or take action in the form of removing a listing or banning a registered user. Reports will be displayed on the Registered Users profile or Listing Page.
    - **Permissions -** The level of access the admin has in managing the site *(mandatory)*
- **Unregistered Users** - Users that have not have created an account on the site
    - **Access Level -** Limited to accessing the homepage, browsing, searching, and filtering listings, and viewing listing details. *(mandatory)*
- **Registered Users** - Users that have signed up on the site using a SFSU verified email
    - **Name -** The user's full name *(mandatory)*
    - **Email -** The email used to sign up for the account. Must be a verified SFSU email *(mandatory)*
    - **Role -** Defines permissions based on whether they are a buyer, seller, or admin *(mandatory)*
    - **Status -** Tracks if the user is active or banned *(mandatory)*
    - **Rating -** Rating of the user given by other users that have interacted with them *(mandatory)*

- **Listing -** An item for sale posted by and belonging to a registered user.
    - **Owner -** The registered user that the listing belongs to *(mandatory)*
    - **Title -** The name of the item for sale. *(mandatory)*
    - **Image** - An image of the item for sale set by the seller. *(mandatory)*
    - **Description -** A description of the item for sale set by the seller. *(mandatory)*
    - **Price** - Price of the item set by the user. *(mandatory)*
    - **Category** - The group of listings it belongs to *(mandatory)*
    - **Tags** - Additional searchable terms set by the seller. *(optional)*
        - **Course** - Sellers can also set an SFSU course as a tag for listings related to a course. *(optional)*
    - **Status -** Shows whether listing is currently being sold or not *(mandatory)*
    - **Date Posted** - The date that the listing was created by the seller
- **Category -** Grouping of a listing provided by the site
    - **Name -** Descriptor of the type of listing *(mandatory)*
- **Tag** - Additional searchable term used to describe listings
    - **Name -** The keyword assigned to a listing *(mandatory)*
- **Request** - Used to show a buyer's interest in a buying a listing
    - **Listing -** The listing the buyer wants to purchase *(mandatory)*

    - **Buyer -** The user sending the request *(mandatory)*
    - **Seller -** The seller of the associated listing *(mandatory)*
    - **Status -** Shows whether the request is pending, accepted, or rejected *(mandatory)*
    - **Sent Date -** The date the request was made *(mandatory)*

- **Messaging** - Allows communication between the seller and the buyer.
    - **Sender** - The user sending the message *(mandatory)*
    - **Receiver -** The user receiving the message *(mandatory)*
    - **Content -** The text sent between users *(mandatory)*
    - **Sent Time -** The time the message was sent *(mandatory)*
    - **Related Request -** The listing request associated with the conversation *(optional)*

- **Notification** - An alert that informs a user of a new message or request
    - **Recipient -** The user receiving the notification *(mandatory)*
    - **Message -** The associated message that triggers the notification *(mandatory)*
    - **Status -** Used to show if the message has been read or not *(mandatory)*
    - **Time Created -** The time the notification was created *(mandatory)*

- **Bought Listing -** A successful transaction between two users
    - **Listing -** The associated item bought *(mandatory)*
    - **Buyer -** The user who purchased the item *(mandatory)*
    - **Meet Up Location -** The recorded location where the two users exchanged goods *(mandatory)*
    - **Purchased Date -** The time the transaction occurred *(mandatory)*

- **Listing Report** - A report sent by a user against a listing that violates the sites guidelines
    - **Reported Listing -** The associated listing that is being reported *(mandatory)*
    - **Reporter -** The user sending the report *(mandatory)*
    - **Reason -** The reason for how the listing violates the sites guidelines *(mandatory)*
    - **Date -** The date the report was sent *(mandatory)*
- **User Report -** A report sent by a user against another user that has violated the sites guidelines
    - **Reported User -** The associated user that is being reported *(mandatory)*
    - **Reporter -** The user sending the report *(mandatory)*
    - **Reason -** The reason for how the listing violates the sites guidelines *(mandatory)*
    - **Date -** The date the report was sent *(mandatory)*

3. Functional Requirements

1 - must have | 2 - desired | 3 - opportunistic

Priority 1

Unregistered User

1. The system shall allow visitors to browse publicly available item listings.
2. The system shall allow visitors to search listings using keywords.
3. The system shall allow visitors to view item details including title, description, price, and images.
4. The system shall allow visitors to filter listings by category, price, and rating.
5. The system shall allow users to register using an SFSU email address.

Registered User

6. The system shall allow users to receive notifications when they receive messages upon the next page refresh.
7. The system shall verify that registered users are associated with an SFSU email domain.

Seller

8. The system shall allow registered users to create listings for items they wish to sell.
9. The system shall allow users to upload images when creating listings.
10. The system shall allow users to mark listings as sold.
11. The system shall allow users to delete their listings.

Buyer

12. The system shall allow users to initiate the sending of messages to other users regarding listings.

Admin

13. The system shall allow administrators to remove listings that violate policies.
14. The system shall allow administrators to monitor marketplace activity.

Priority 2

Registered User

Seller

15. The system shall allow users to edit their listings.

Buyer

Admin

16. The system shall allow administrators to manage listing categories.
17. The system shall allow administrators to suspend users who repeatedly violate rules.

Priority 3

Registered User

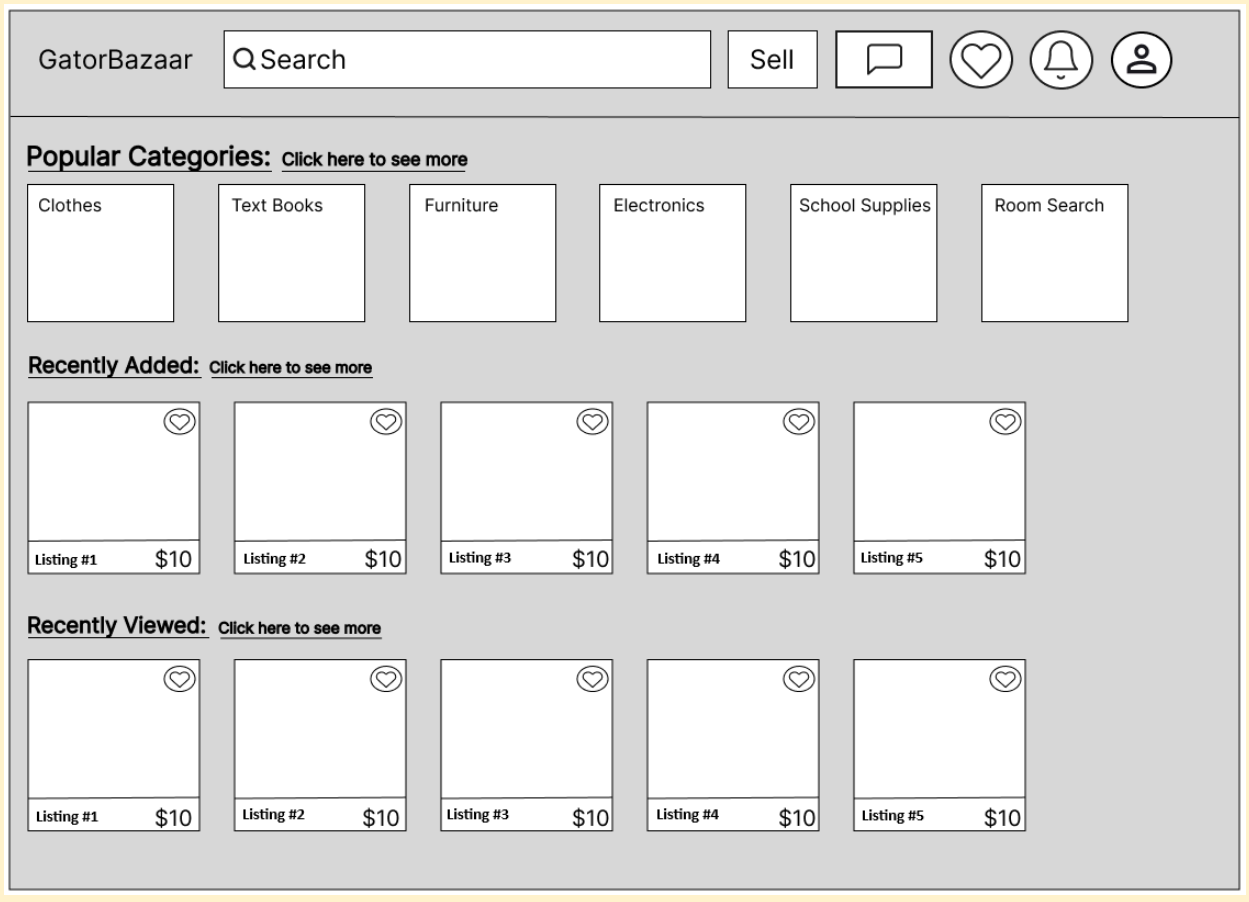
18. The system shall allow users to report listings that violate platform rules.
19. The system shall allow users to give a rating to either seller or buyer.

Admin

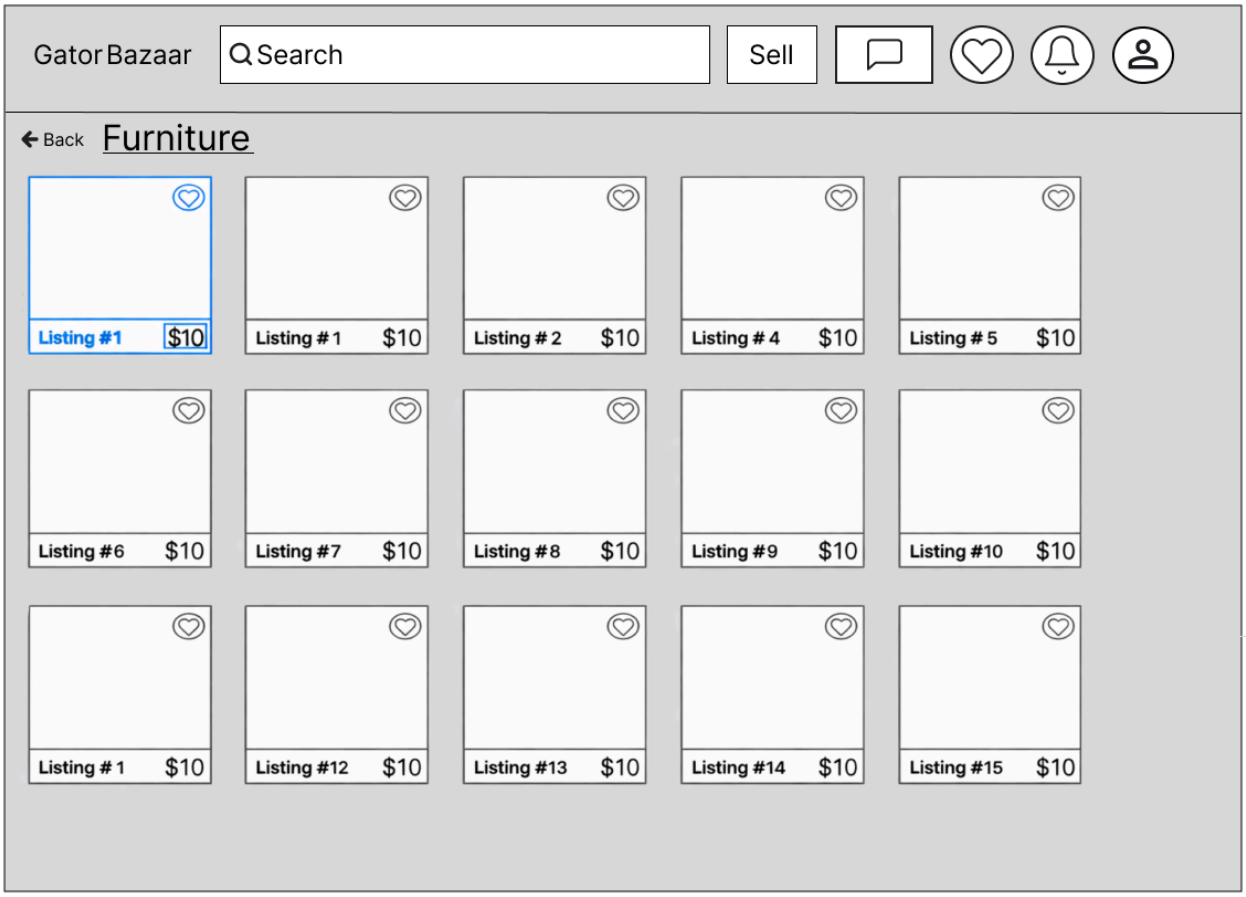
20. The system shall allow administrators to review reported listings.

4. UI Storyboards

Use Case 1: Buying Furniture in GatorBazaar



Landing on the home page of our application, John navigates to the furniture tab located in Popular Categories at the top of the page.

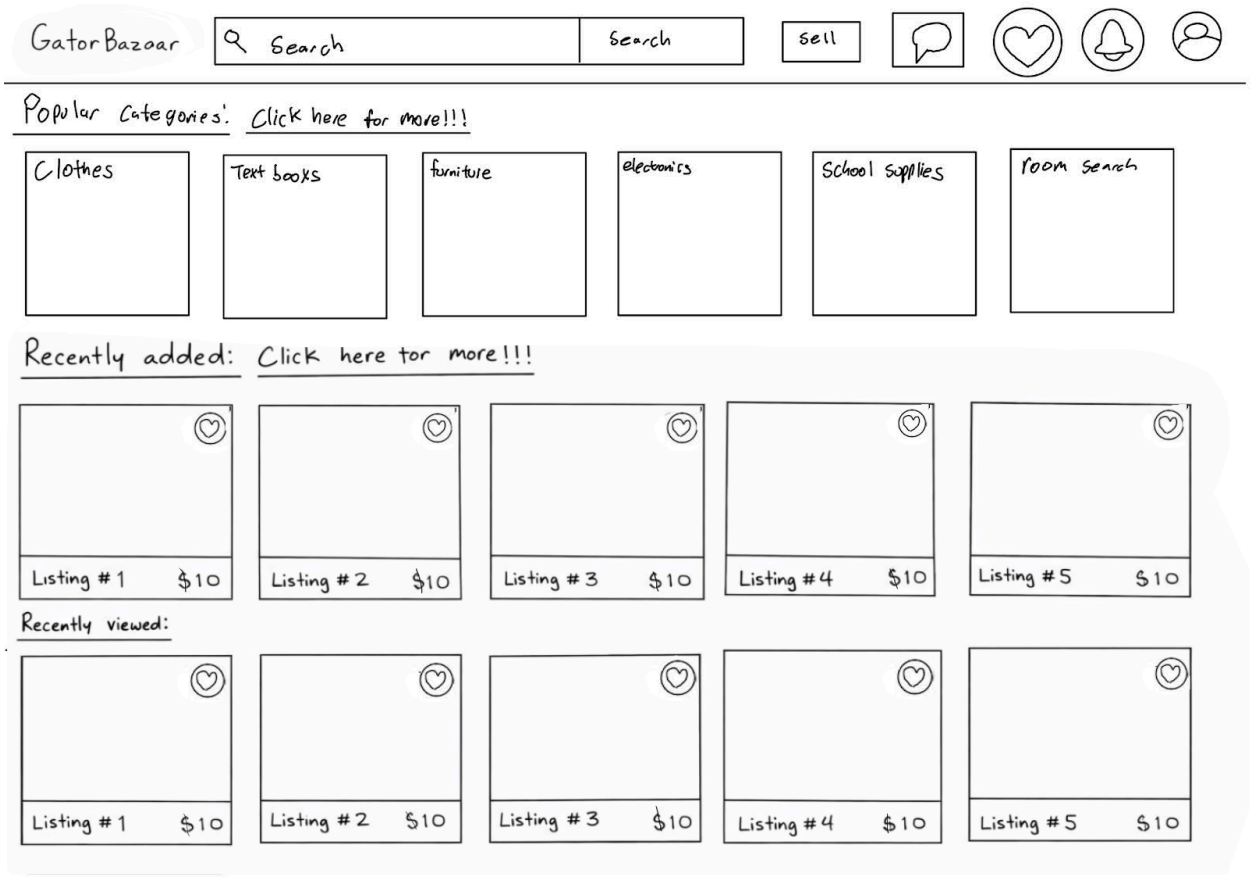


Then he can start scrolling through the page to find lists of furniture he needs.

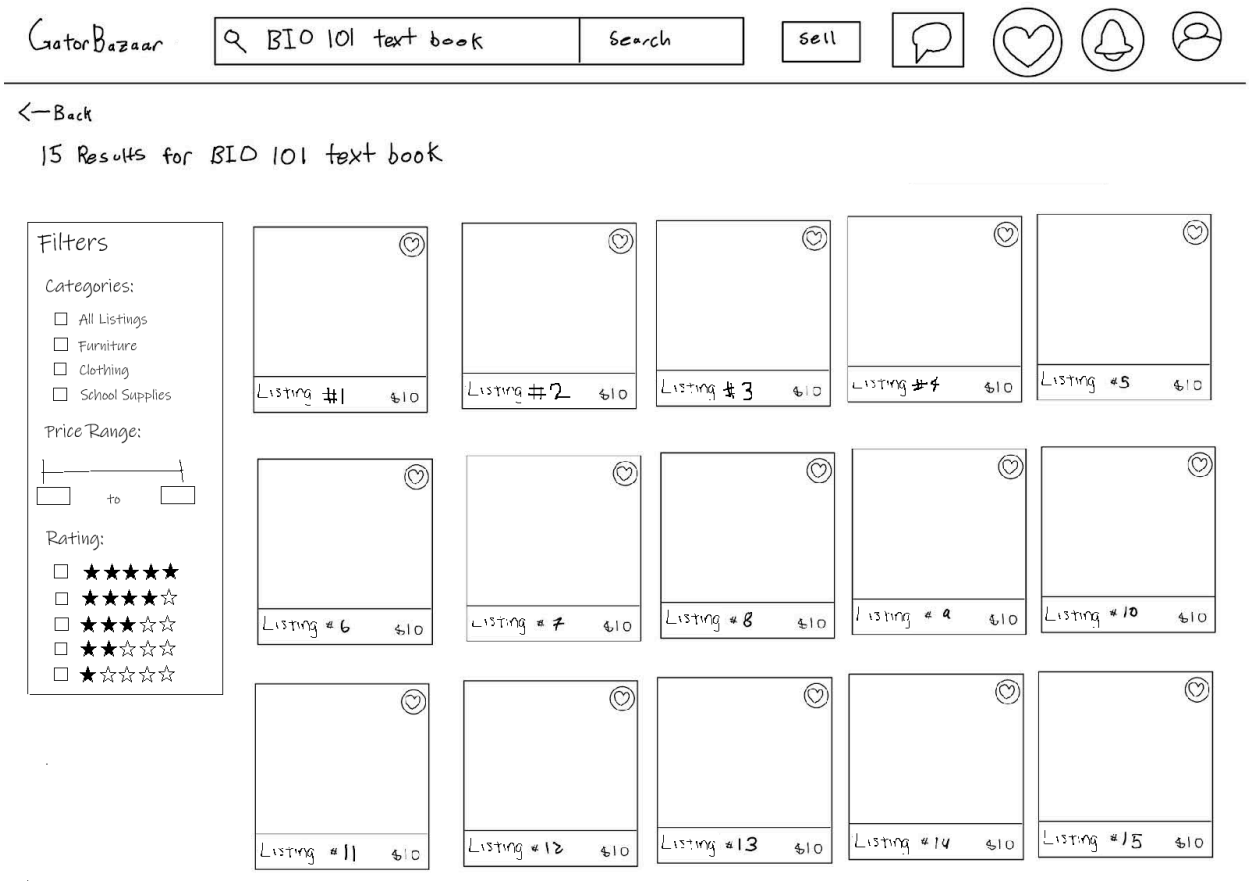


Once he clicks on a furniture listing he likes, he will be brought to the 'buying' tab where he can find the description, and other useful images of the furniture from the seller. He can either like to save the furniture or message the seller to buy the furniture from the seller at the top right of the listing.

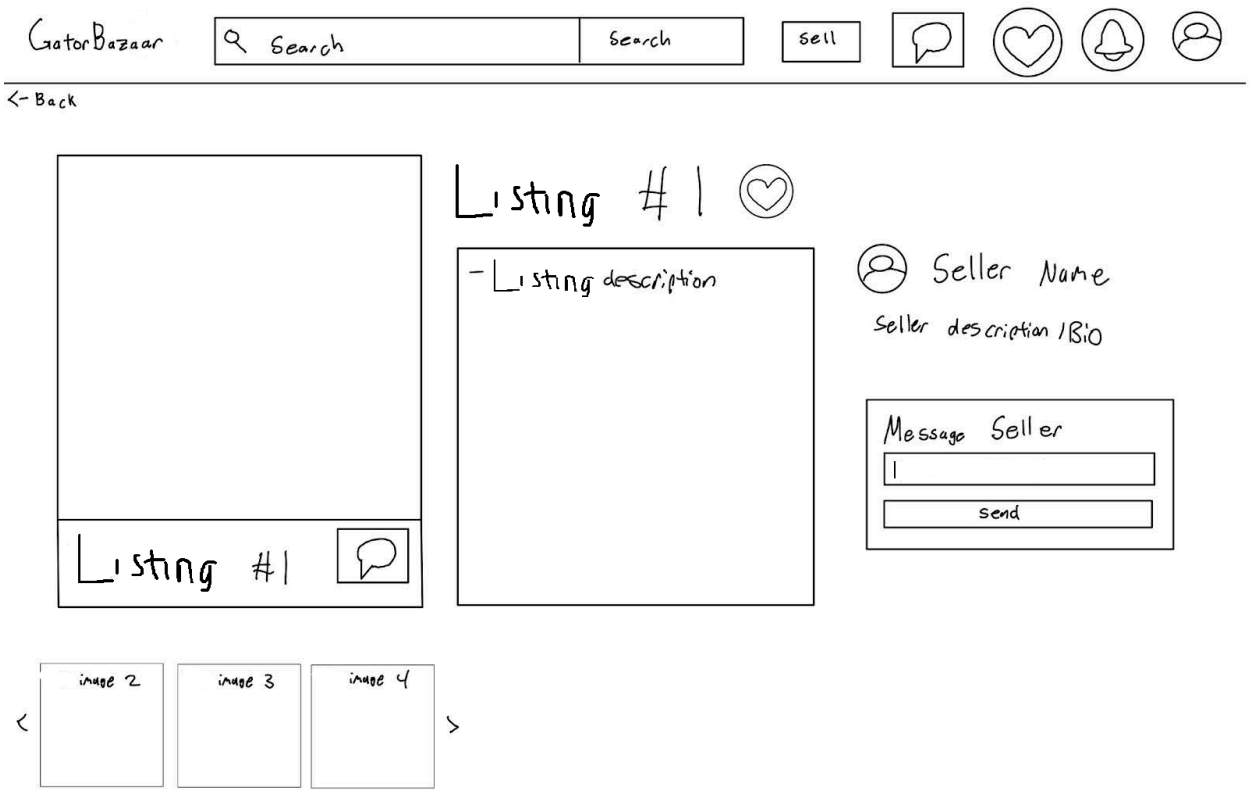
Use Case 2: Buying textbooks



Jose is looking to buy some textbooks for his classes, so he logs into GatorBazaar and searches for his Bio 101 textbook.

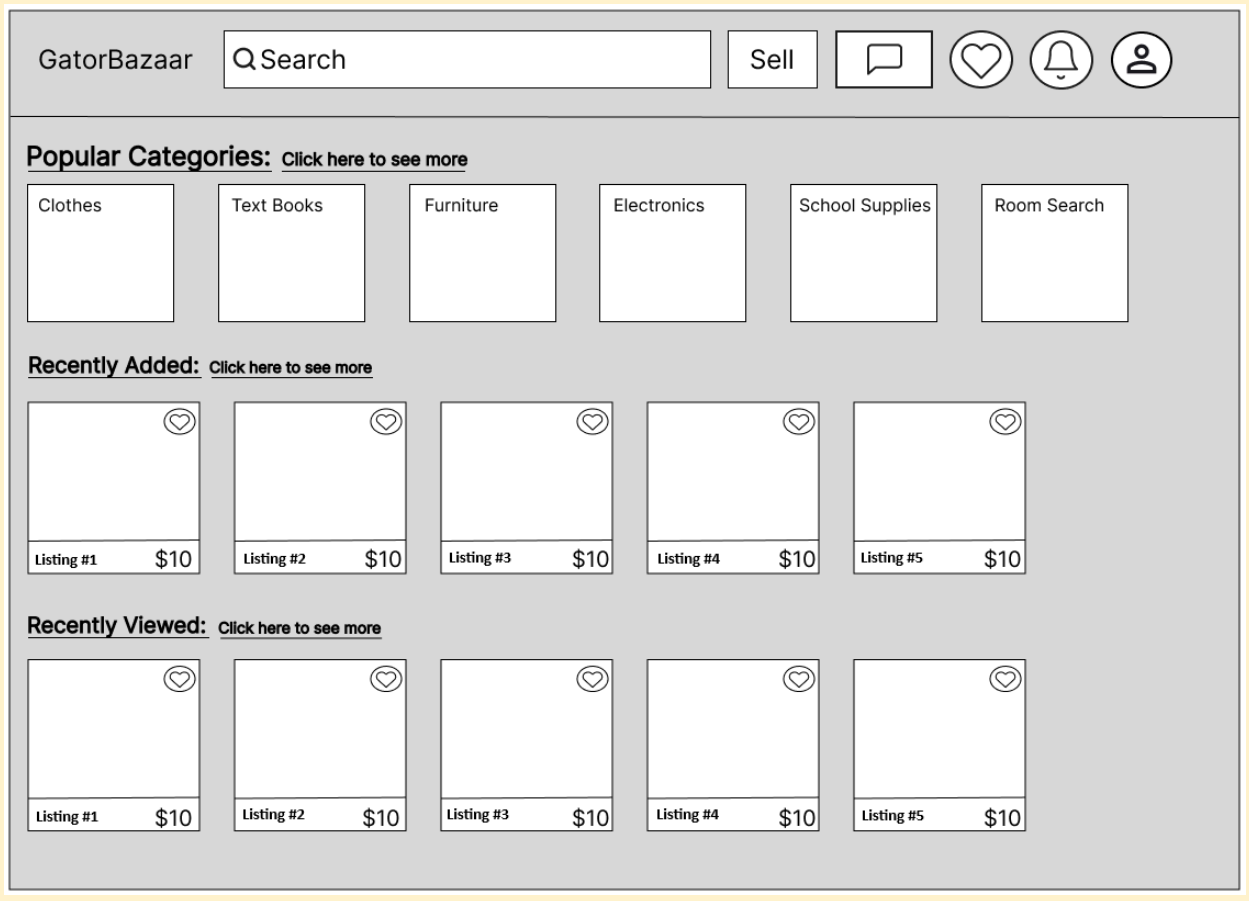


Once Jose searches for his textbook, he is prompted with the closest results to his given search. And because he added BIO 101 to his search, items with the course tag BIO 101 are displayed more often.

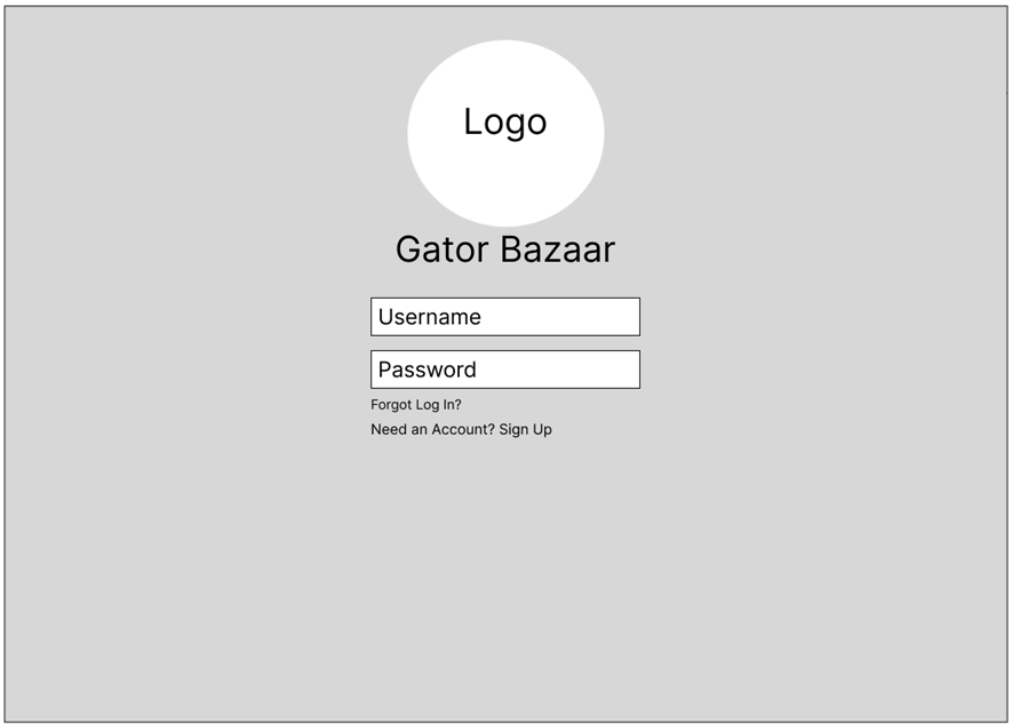


Once he selects a listing, Jose can see the seller's item description, multiple images, and the seller's bio. In order to be able to buy an item, Jose has to get in touch with the seller by sending them a message using the Message Seller box.

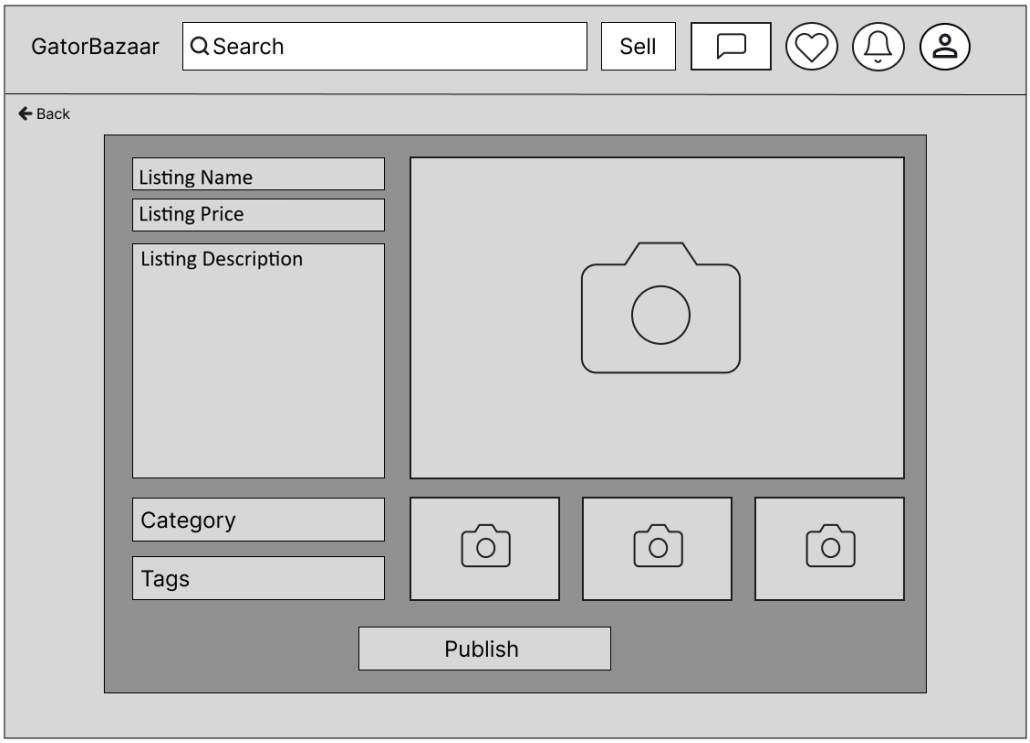
Use Case 3: Selling Items in GatorBazaar



Javier wants to sell some of the things he does not need before moving out of his apartment. However, before he can sell items in GatorBazaar, he needs an SFSU-verified account using his SFSU email, which is the user log-in located at the top right of the page.

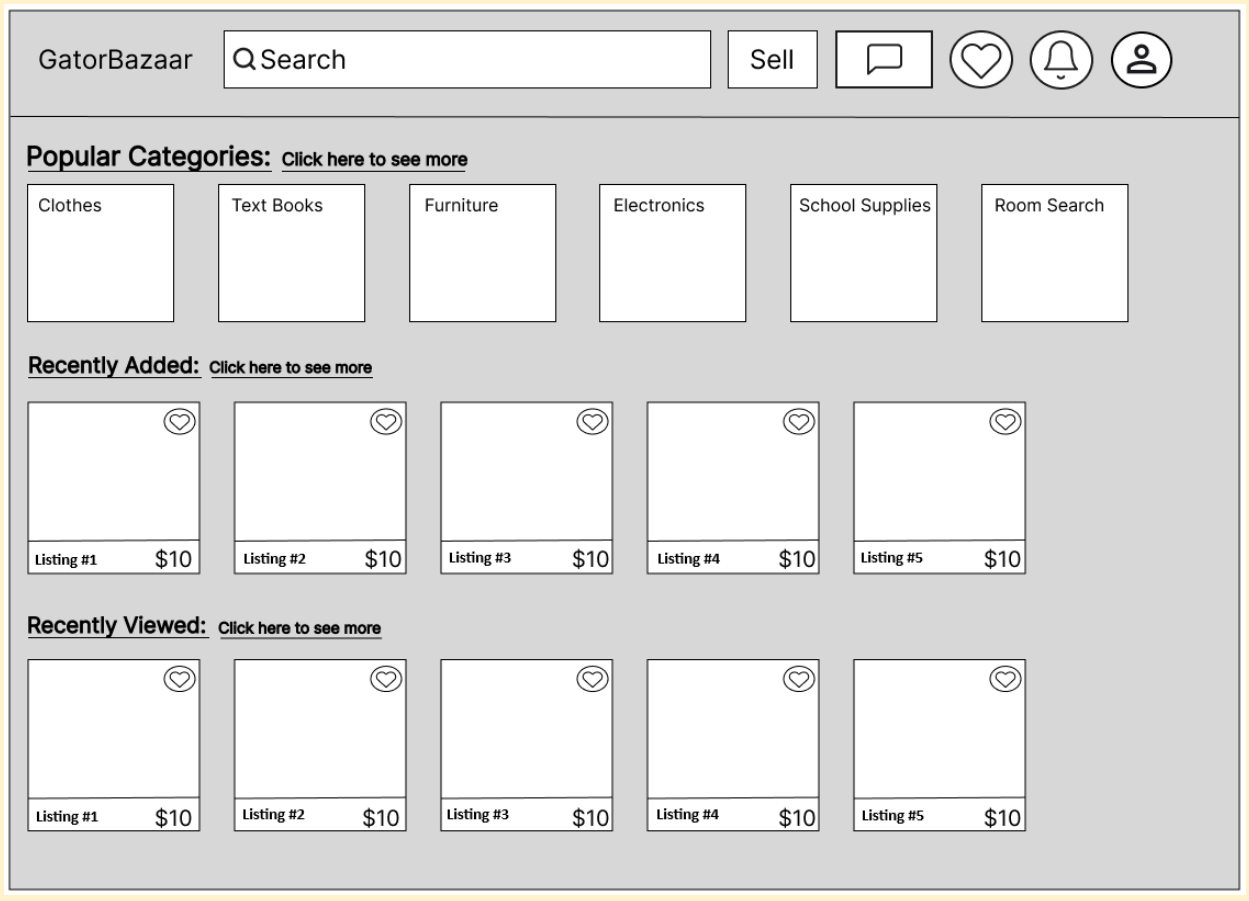


At the login page, he can sign in if he has an account set up, but he doesn't have one, so he needs to sign up with his SFSU verified email. Then he can log in with that account to start selling.

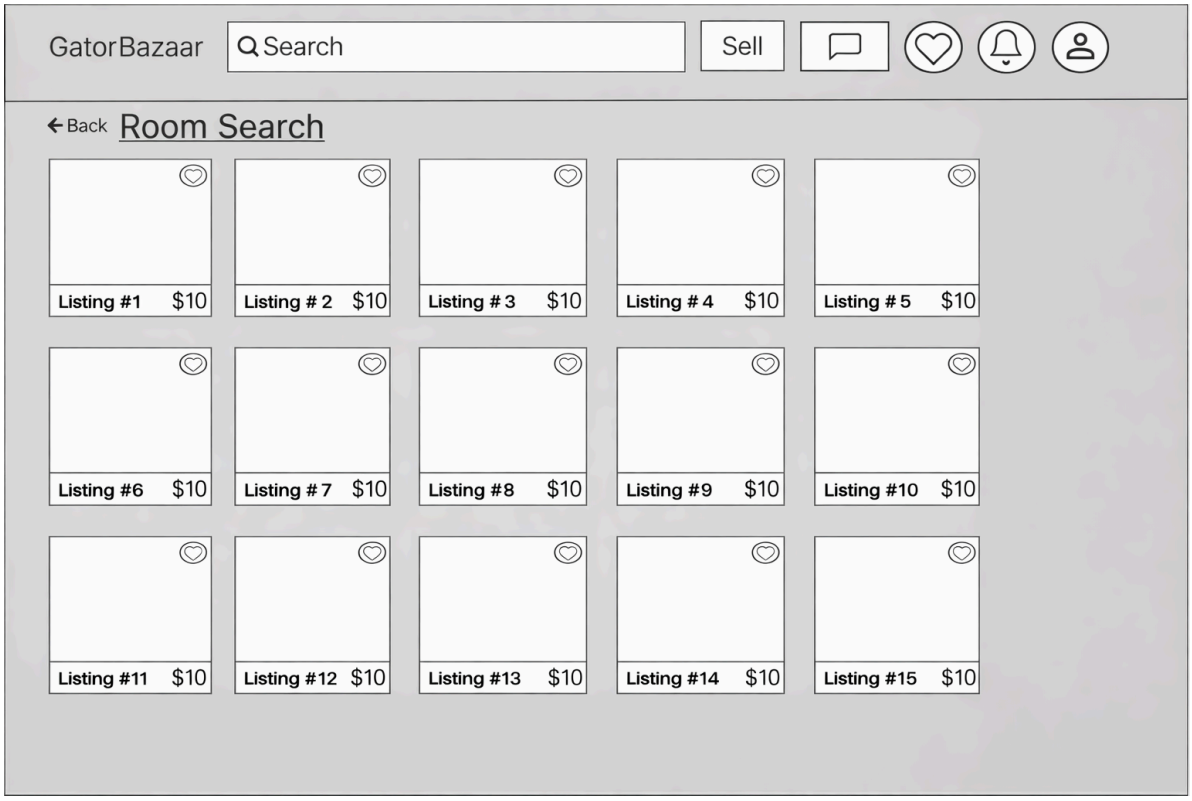


Once he signs up, he can click on the sell button at the top of the page to start selling. In the selling page, he can put the name, what type of item he's selling, the description, the location of the item, and other filters or necessary information he wants, and also upload necessary pictures of the item. When he is finished, he can press the publish button to publish the listing to GatorBazaar.

Use Case 4: Finding an Apartment in Gator Bazaar



Jose plans to move out of his dorm and live in an apartment, however he wants to cut the cost of the rent by finding a roommate. He uses Gator Bazaar to find apartment listings available in his area. At the home page, he clicks the room search category to filter out his search to apartments only.



He browses through different listings to find apartments that fit his liking. When he finds an apartment he likes he can click on it to see the listings' description.



Once he finds the apartment to his liking, he can see various information like the seller's contact, the listing description, and pictures. He can then message the seller using the message button at the top of the listing to ask more about the listing.

Use Case 5: User Admin

No mockup for the Admin UI will be displayed here as our team will be using Workbench.

5. High Level Architecture/Database Organization:

Database Organization

Architecture Pattern (MVC)

The application follows the Model-View-Controller (MVC) pattern:

- Model:
  MySQL database schema and backend data logic
- View:
  React.js frontend interface rendered in the browser
- Controller:
  Node.js / Express.js backend handling API routes, business logic, and database interaction

Search/filter architecture and implementation

Our search Algo will be SQL-based keyword search %Like. Queries will be done through node.js/expressbackend. An example will consist of **WHERE title LIKE '%keyword%'** LIKE operator. Searchable fields consist of LISTINGS: title, description, category, course - TAGS tag_name. No current plan on switching from the recommended LIKE operator.

Registered Users Tables

- userid(Primary Key)
- Email
- Password_hash
- role(buyer-seller/admin)
- Created_at
- User_rating
- is_suspended

Listings Table

- listingid (Primary Key)
- Userid (Foreign Key -> registered users)
- Title
- Description
- Price
- Category
- active(boolean)

- created_at

Listing Images Table

- Image_id (primary Key)
- Listing_id (foreign Key -> Listings)
- Image_url

Tags Table

- Tag_id (Primary Key)
- Tag_name

Listing Tags Table

- Listing_id (Foreign Key-> Listing)
- tag_id (Foreign Key → Tags)

Sent Requests Table

- request_id (Primary Key)
- listing_id (Foreign Key → Listings)
- buyer_id (Foreign Key → Registered Users)
- seller_id (Foreign Key → Registered Users)
- status (pending / accepted / rejected)
- created_at

Messages Table

- message_id (Primary Key)
- request_id (Foreign Key → Sent Requests)
- sender_id (Foreign Key → Registered Users)
- receiver_id (Foreign Key → Registered Users)
- message_content
- sent_at

Notifications Table

- notification_id (Primary Key)
- user_id (Foreign Key → Registered Users)
- message_id (Foreign Key → Messages)
- is_read (boolean)
- created_at

Bought Listings Table

- purchase_id (Primary Key)
- listing_id (Foreign Key → Listings)
- buyer_id (Foreign Key → Registered Users)
- purchase_date

Listing Reports Table

- report_id (Primary Key)
- listing_id (Foreign Key → Listings)
- reported_by_user_id (Foreign Key → Registered Users)
- reason
- created_at

User Reports Table

- report_id (Primary Key)
- reported_user_id (Foreign Key → Registered Users)
- reported_by_user_id (Foreign Key → Registered Users)
- reason
- created_at

Categories Table

- category_id (Primary Key)
- category_name

Non-Trival Algorithms / Processes

By default, listings will be sorted by the time they were posted with the most recent listings appearing first. At the results page, users can further sort listings by price or the rating of the listing's seller. These attributes will be pulled from the resulting listings once the search is entered.

6. Key Risks

1. Skills Risk

Some team members may have a limited amount of experience with full-stack development in a group project. Particularly with integrating the React front-end with the Node.js/Express backend and MySQL database. This could slow development and lead to implementation issues and troubles.

Mitigation:

Team members will divide work based on strengths (front-end & back-end) and actively learn missing skills through documentation, tutorials, and help from other team members. Regular check-ins and reviews will also help to ensure that knowledge is properly shared and understood across the whole team.

2. Schedule Risk

The project includes multiple features such as listings, messaging, user ratings, and admin controls, which may be difficult to complete within the given timeline if not managed properly.

Mitigation:

Our team will prioritize core functionality first and treat additional features as secondary. Tasks will be tracked using ClickUp with clear deadlines and responsibilities to mitigate potential issues whenever possible.

3. Technical Risk

Handling image uploads for listings may introduce some challenges not originally prepared for such as storage limitations or potentially slow loading times with multiple images.

Mitigation:

Our team will use good practice techniques in order to handle image handling issues, and possibly use other storage solutions if needed. Along with image size limits that will be enforced in order to keep storage issues to a minimum.

4. Teamwork Risk

Lack of clear communication regarding feature requirements may result in inconsistent implementation across components.

Mitigation:

The team will document requirements clearly and discuss them during meetings. Shared documentation will be maintained to ensure all team members understand what is required of a feature.

5. Legal / Content Risk

Users may post inappropriate or prohibited items (copyrighted material, or otherwise prohibited content), which could create compliance or safety concerns, and open the platform up to legal action if not dealt with.

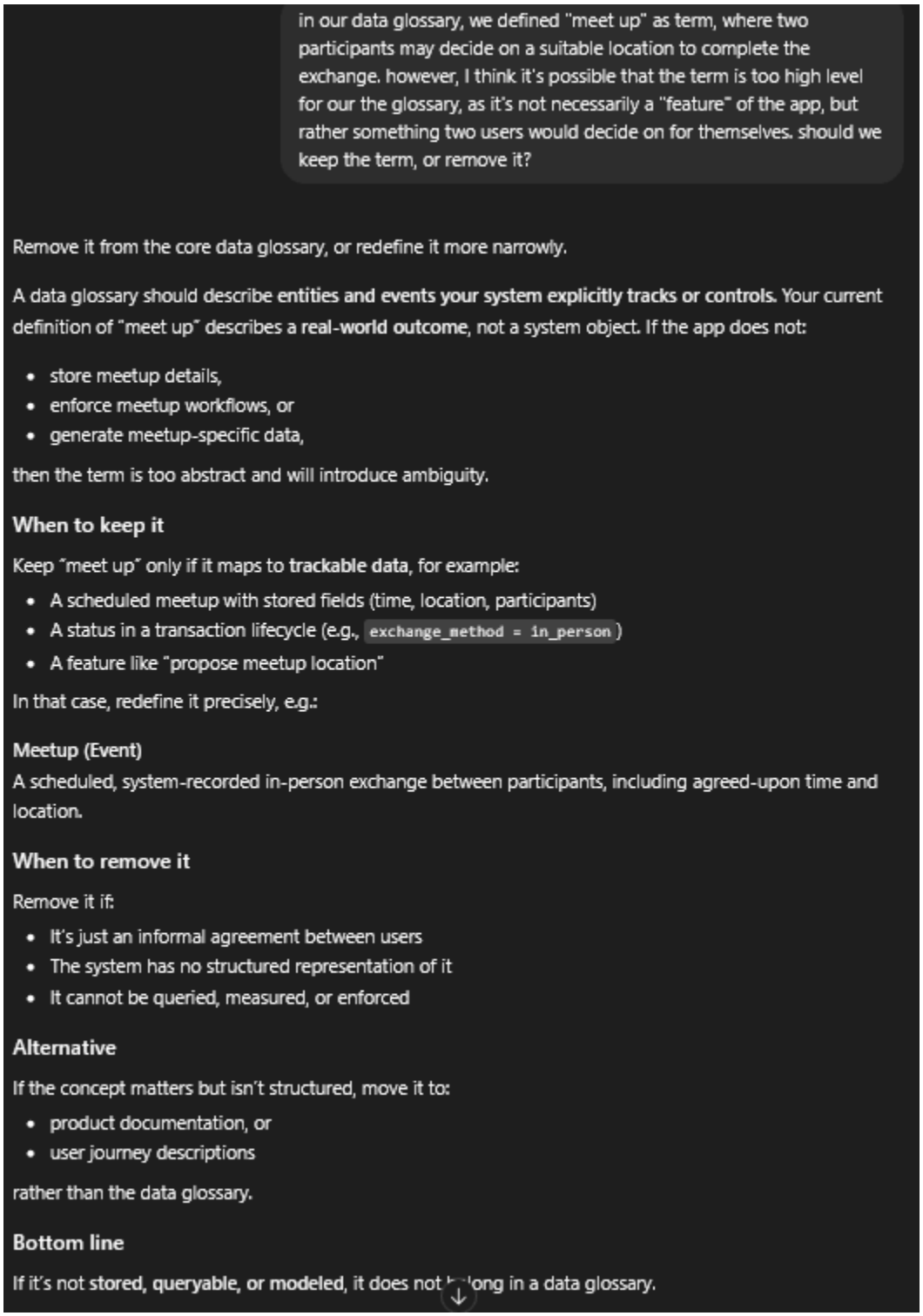
Mitigation:

The system will include reporting functionality and admin moderation tools to remove inappropriate listings. Clear platform guidelines will be established, and admins will actively monitor reported content.

7. Project Management

Our process for task management in Milestone 2 was to separate Part I and II, and then divide the deliverables among the team. When creating our tasks, we initially separated the M2 Part I deliverable by section to get a good idea of what could be worked on independently. Then, we looked through each section to see what could be left as a whole task and what needed to be divided among multiple team members or divided into multiple tasks. While team members have input into what tasks they're assigned, we noted that certain tasks would be better off assigned to the front-end or back-end team. An example of this is the UI Storyboards, which were split among the front-end, or the Database Summary, which was assigned to the back-end lead. For Part II, we plan on taking the suggestion of sacrificing a front-end and back-end member to build the prototype and web pages, but we also emphasized that while only two members might be working on the Vertical Prototype, because we're defining the architecture and how the front-end and back-end will integrate, all team members must keep an eye on the process. We landed on a task due date of two days before the deadline for Part I and a week later for Part II, giving us sufficient time for revisions and peer reviews. Three days before the task deadline, we intend to have a checkup meeting where we can get a feel for where we are in terms of progress, with members already having posted occasional updates on their tasks.

8. Use of GenAI Tools:



in our data glossary, we defined "meet up" as term, where two participants may decide on a suitable location to complete the exchange. however, I think it's possible that the term is too high level for our the glossary, as it's not necessarily a "feature" of the app, but rather something two users would decide on for themselves. should we keep the term, or remove it?

Remove it from the core data glossary, or redefine it more narrowly.

A data glossary should describe **entities and events your system explicitly tracks or controls.** Your current definition of "meet up" describes a **real-world outcome**, not a system object. If the app does not:

- store meetup details,
- enforce meetup workflows, or
- generate meetup-specific data,

then the term is too abstract and will introduce ambiguity.

When to keep it

Keep "meet up" only if it maps to **trackable data**, for example:

- A scheduled meetup with stored fields (time, location, participants)
- A status in a transaction lifecycle (e.g., `exchange_method = in_person`)
- A feature like "propose meetup location"

In that case, redefine it precisely, e.g.:

Meetup (Event)
A scheduled, system-recorded in-person exchange between participants, including agreed-upon time and location.

When to remove it

Remove it if:

- It's just an informal agreement between users
- The system has no structured representation of it
- It cannot be queried, measured, or enforced

Alternative

If the concept matters but isn't structured, move it to:

- product documentation, or
- user journey descriptions

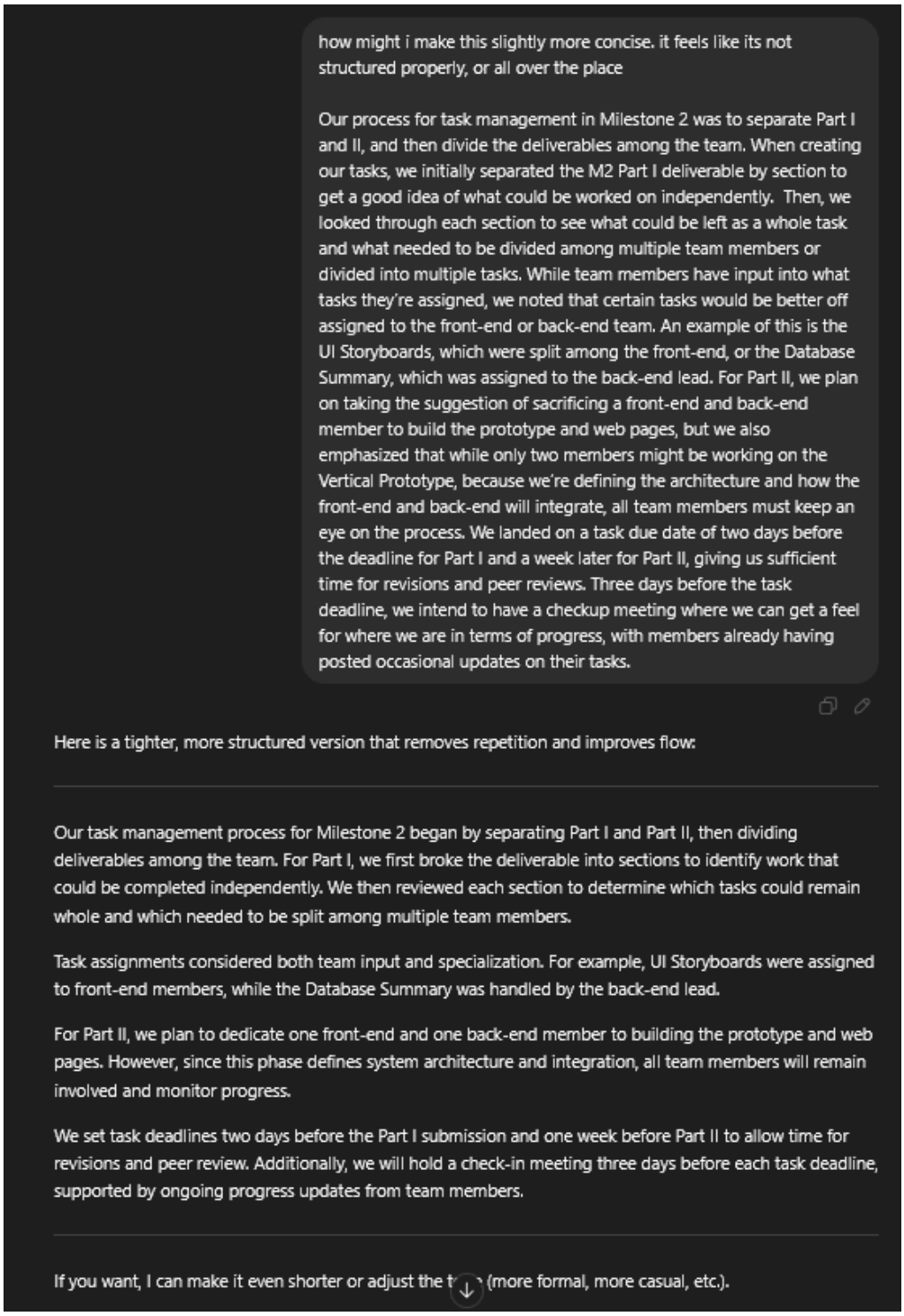
rather than the data glossary.

Bottom line

If it's not **stored, queryable, or modeled**, it does not belong in a data glossary.

Medium Usage: We used it to get a second opinion on the necessity of the meetup term in the data glossary, as we were having second thoughts about whether it was needed, since in our

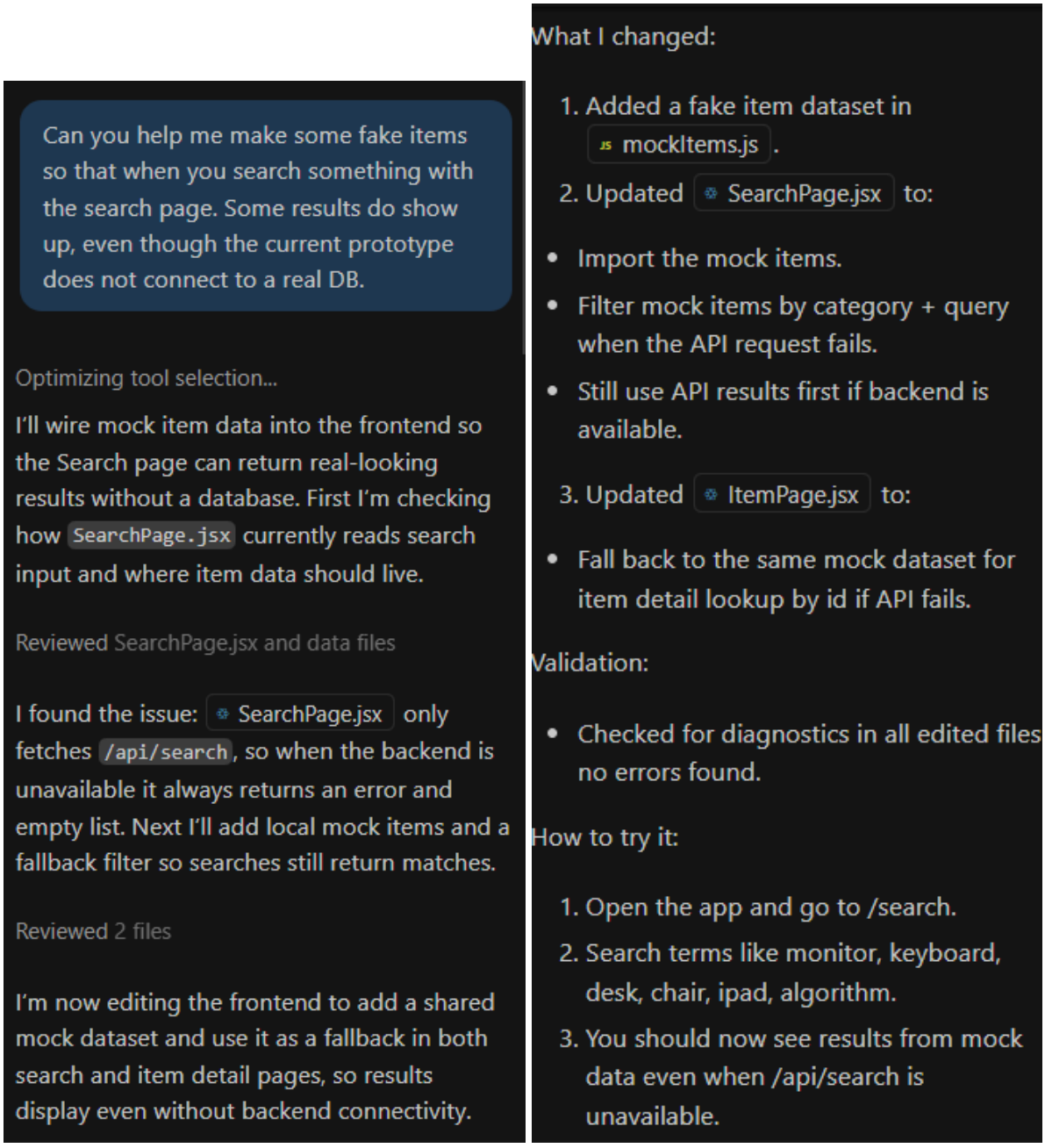
initial design it wasn't explicitly part of the website functionality but something two users would decide on for themselves. From these comments we adjusted our implementation to add the meetup location as a field that the users input and is tracked for the exchange.



Low Usage: We reworded the project management paragraph. We wanted to see if we should reword the project management paragraph, as it might be a bit too long or could be better

structured. After reviewing the output from ChatGPT, we ultimately decided that the current iteration was good enough, and that there wasn't a need to make it more concise or change things up too much. After the results, we ended up preferring the previous version because of better structurization and explanation.

Prompt: Given the above project instructions, please make the frontend of the vertical prototype, make sure to make it as a real part of the application and explain the thought process. Make sure to follow the instructions and requirements listed. Making sure to create a Search and Result Page with categories such as Books, Electronics, and Furniture.
Medium Usage: We used Github's Copilot, running ChatGPT 5.2 to process this prompt. We primarily used the tool to get a very basic Frontend Vertical Prototype built in React very quickly, so that we could look at it and modify it ourselves without having to waste time initially writing the bones of the webpage for the prototype. After feeding it the prompt above and the FrontEndVP instructions from Milestone 2 we received a nice Search and Result page from the AI, and used it as a basis for the actual Frontend Vertical Prototype that we decided on and hooked to the Backend. We had to modify some routes and rewrite/fix some bugs such as routes not working, images clipping, and adding some features to make the UX better such as auto-focus for the search bar, when loading and changing categories.



High Usage: We used Github's Copilot, running ChatGPT 5.2 to process this prompt. We used it to make some mock items to have in the 'database' so that we could test the Frontend Vertical Prototype without having to connect it to a database. It ended up giving us an array of JSON objects that fit our criteria and allowed us to test our Search and Results page. It also decided to make the mock items a fallback to the originally implemented API search, so that if nothing returned from the API we could still test the Frontend. Overall the mock items were great and extremely easy to use and modify to test our Frontend, although we did eventually have to rewrite some parts of the Search Page's api call when we connected it to the database.

9. Team Lead Check List:

Check List	
Done	So far all team members are fully engaged and attending team sessions when required
Done	Team ready and able to use the chosen back and front end frameworks and those who need to learn are working on learning and practicing
Done	Team reviewed suggested resources before drafting Milestone 2
Done	Team lead checked Milestone 2 document for quality, completeness, formatting and compliance with instructions before the submission
Done	Team lead ensured that all team members read the final Milestone 2 document and agree/understand it before submission
Done	Team shared and discussed experience with GenAI tools among themselves

M3 Feedback Summary From In-Person Review Meeting:

Team number: 7
Meeting date: April 28th

Summary of feedback on UI (record all pages that need revision)

The UI feedback primarily focused on improving layout consistency, spacing, responsiveness, and visual polish across multiple pages. On the home page, category boxes were considered too large for short labels, creating excessive dead space, and categories should be more compact and mobile-friendly. A footer is currently missing and should be added. The top banner should only contain the members button in the top-right corner, and the home button on the search page needs stronger contrast. Rounded corners were recommended for category and dropdown elements for visual consistency.

Several page-specific revisions were suggested. On listing-related pages, heart/favorite icons should use a more visually consistent design, and visually favoriting an item is sufficient feedback for the user. The create listing page should be centered with improved padding, image upload placement should not appear at the bottom of the form, and multiple image previews should be supported without images dominating the layout.

Inbox and messaging flow also received feedback. Users should not automatically redirect to chat after sending a message from a listing page, but conversations should automatically select when entering the inbox. Loading states should be improved using React Suspense because the account page sometimes fails to detect the user during the first render.

The transaction completion page should visually indicate whether the buyer or seller has completed their confirmation using a checkmark or green circle indicator. Additionally, the team member page does not need the full navbar or dashboard navigation.

Mobile responsiveness also needs revision. Category layouts break on smaller screens and consume too much space, although implementing a hamburger menu is optional. The color scheme was positively received overall and considered consistent.

Summary of feedback on code and architecture

The instructor noted that the overall application structure is solid and the search functionality works correctly during testing. However, there are several architectural concerns to address. The .env file was accidentally committed to the repository, which poses a security risk and should be removed from version control immediately.

Another major concern is backend startup reliability. If the database connection fails, the server still starts and silently fails on incoming requests. The instructor recommended implementing a fast-fail startup strategy so the server does not continue running when database connectivity is unavailable.

Additional frontend architecture feedback included improving loading behavior with React Suspense and reducing layout fragility where changing element heights currently breaks panel layouts.

Summary of feedback on GitHub usage

The GitHub workflow feedback focused on repository organization and commit hygiene. Branch naming should follow a more consistent convention such as prefixes indicating a feature or a fix. There are also many remote branches that have been merged, delete them to reduce clutter.

Commit messages also need improvement because several commits use vague or duplicate descriptions. While the current commit counts were considered acceptable, the instructor expects stronger collaboration and a more even distribution of contributions as the project approaches demo day.

Summary of feedback on DB

The database itself was reported as functional and accessible during review, and SSH/database access were both confirmed to work correctly.

However, there were concerns regarding backend/database interaction reliability. Specifically, the server currently continues running even when the database connection fails, causing silent failures during requests. The instructor recommended implementing proper startup validation and preventing the application from launching unless the database connection succeeds.

Summary of feedback on teamwork and risk management

The instructor reviewed contribution distribution using commit counts and noted that participation levels are acceptable for now, though collaboration expectations will increase as the team approaches demo day.

Risk management concerns mainly focused on repository maintenance and operational stability. Risks identified include:

- accidental exposure of sensitive .env files,
- excessive stale Git branches,
- inconsistent commit practices,
- and the backend's failure to stop when the database is unavailable.

Architecture Review:

- App structure is fine, Github branches need to be cleaned up and commit messages should be more detailed. Commit counts okay for now but can be improved on.

P1 Features:

Priority 1:

- Unregistered User
  - The system shall allow visitors to access the marketplace homepage.
  - The system shall allow visitors to browse publicly available item listings.
  - The system shall allow visitors to search listings using keywords.
  - The system shall allow visitors to view item details including title, description, price, and images.
  - The system shall allow visitors to filter listings by category.
- Registered User
  - The system shall allow users to register using an SFSU email address.
  - The system shall allow users to receive notifications when they receive messages.
  - The system shall verify that registered users are associated with an SFSU email domain.
- Seller
  - The system shall allow registered users to create listings for items they wish to sell.
  - The system shall allow users to upload images when creating listings.
  - The system shall allow users to mark listings as sold.
  - The system shall allow users to delete their listings.
- Buyer
  - The system shall allow users to initiate the sending of messages to other users
  - regarding listings.
- Admin

- The system shall allow administrators to remove listings that violate policies.
- The system shall allow administrators to monitor marketplace activity.

After thorough review of feedback we have made all successful changes from time of M3 feedback to demo day from Professor.

SW Engineering CSC648-848 Spring 2026

GatorBazaar

Team 07

Members	
Team Lead	Dylan Cordoni (dcordoni@sfsu.edu)
Github Maintainer	Abhay Bajaj
Back-End Lead	Sebastian Cervantez
	Ryan Zhang
Front-End Lead	Daniel De Jesus
	Kenny Camarra
	Diego Rivera-Soto

Milestone 4

History Table	
Date Submitted	May 20, 2026

1. Product Summary

- **Product Name:** GatorBazaar
- **Describe your product:** GatorBazaar provides a simple web-based platform where users can list items they no longer need and browse items posted by others. Listings are organized into categories such as textbooks, electronics, clothing, and dorm essentials, and many other features we have coming down the pipeline, making it easy for users to quickly find what they are looking for. Students can post items for sale, upload descriptions and prices, and connect with buyers within the SFSU community. Professors can also use the platform to share or sell academic materials or recommend resources to students. By centralizing these exchanges in one place, GatorBazaar helps reduce waste, saves students money, and creates a convenient and safe way to reuse items within the campus community.
- **Itemized list of ALL major committed functions (your final priority 1 or P1 list):**
  Priority 1:
    - Unregistered User
        - The system shall allow visitors to access the marketplace homepage.
        - The system shall allow visitors to browse publicly available item listings.
        - The system shall allow visitors to search listings using keywords.
        - The system shall allow visitors to view item details including title, description, price, and images.
        - The system shall allow visitors to filter listings by category.
    - Registered User
        - The system shall allow users to register using an SFSU email address.
        - The system shall allow users to receive notifications when they receive messages.
        - The system shall verify that registered users are associated with an SFSU email domain.
    - Seller
        - The system shall allow registered users to create listings for items they wish to sell.
        - The system shall allow users to upload images when creating listings.
        - The system shall allow users to mark listings as sold.
        - The system shall allow users to delete their listings.
    - Buyer
        - The system shall allow users to initiate the sending of messages to other users
        - regarding listings.

    - Admin
        - The system shall allow administrators to remove listings that violate policies.
        - The system shall allow administrators to monitor marketplace activity.
- **URL to your product accessible to instructors:** http://3.150.110.210

2. Usability test plan for selected function

1. Test objectives: - The purpose of this usability is to evaluate how users can search for and browse marketplace listings within the web application.. The test focuses on the search bar, category filtering, and viewing item listings. The objective is to determine if the users can easily find items of interest, understand the UI, and navigate with ease.

2. Test background and setup

System Setup:

The system being tested is from our web application marketplace. Developed for students to buy, sell, and browse different kinds of items such as textbooks, furniture, electronics, school supplies, and apartment rooms search. The tester will access the application through a web browser of their choice.

Staring Point:

The tester will begin on our homepage when they visit our website. The tester should not be required to have an account to browse through the marketplace, however would be required to Log in if the tester wants to purchase an item.

Hardware Requirements:

The tester should have a stable internet connection, a laptop or desktop computer, keyboard and mouse, and a web browser installed.

Intended Users:

The intended users are SFSU students who want to browse, search, buy, or sell second-hand items within their campus community. Users who have varying levels of technical experience may use our web application.

URL of the System:

http://3.150.110.210

Test Environment:

The usability test will be conducted remotely from the tester's home environment. No cameras or screen recording are required. The tester may be monitored. The tester will not receive training before beginning the test. We will collect data after the tester completes their test.

3. Usability Task description:

The tester should complete the following tasks in order and must read all instructions carefully before beginning.

Task 1 - Search for an item

1. Search an item
2. Review the search results.
3. Find the item you would consider purchasing

Task 2 - Browse by Category

1. Change the category of your preferred item
2. Go through different categories
3. Click on the item to review

Task 3 - Review Listing Information

1. Find the item price
2. Determine the item condition or description
3. Message the seller

4. Plan for evaluation of Effectiveness:

Effectiveness will be measured by observing whether users can successfully complete the assigned tasks without assistance. The evaluation will focus on whether users can locate items, browse categories, and access listing information correctly. User errors and failed task attempts will also be recorded during testing. A task will be considered effective if the users can complete it successfully without error.

5. Plan for Evaluation of efficiency:

Efficiency will be evaluated by measuring how quickly users are able to complete the assigned tasks while using the web application marketplace. The evaluation will consider the amount of time required to locate items, browse listings, and access listing information. The number of unnecessary actions, repeated attempts, and navigation mistakes made by testers during testing will also be recorded. A task will be considered efficient if the user can complete it smoothly with minimal effort.

6. Plan for Evaluation of user satisfaction (Likert scale questionnaire):

Statement	Strongly Disagree	Disagree	Neutral	Agree	Strongly Agree
The search function was easy to use	1	2	3	4	5
The category navigation	1	2	3	4	5

was clear and understandable					
I was able to find item information quickly and easily	1	2	3	4	5
I would feel comfortable using this marketplace application regularly	1	2	3	4	5

7. GenAI use: Did not use

3. QA test plan and QA testing

1. Test objectives: - The objective of this QA testing is to verify that the marketplace search and browsing functionality operates correctly across different web browsers. The testing focuses on ensuring that users can successfully search for listings, browse categories, and view listing details without functional errors or broken navigation.

2. HW and SW setup (including URL):

HW Setup:

1. Laptop or desktop computer
2. Keyboard and mouse
3. Stable internet connection

SW Setup

1. Operating Systems: Windows/MacOs/Linux
2. Browser 1: Google Chrome
3. Browser 2: Microsoft Edge
4. Local development server running Vite

URL:

http://3.150.110.210

3. Feature to be tested

1. Search bar functionality
2. Category Filtering

3. Item listing display
4. Navigation to item details page

4. QA Test plan: in table format: This is the plan to be given to QA tester to execute your QA test plan. Contains min of 3 test cases and results of testing them on your system: appr. 1 page. You must provide QA test plan in a separate section in the easy-to-read tabular form allowing easy reading and analysis by management e.g. like presented in the class slides on SW QA and outlined below.

Test	Title	Description	Input	Expected correct output	Chrome result	Edge result
1	Search for item	Verify that users can search for listings using keywords	Enter “textbook” into search bar	Relevant textbook listings are displayed	PASS	PASS
2	Category filtering	Verify that category buttons correctly filter listings	Select “furniture” category	Only furniture relevant items are displayed	PASS	PASS
3	View item listing	Verify that users can open and view listing details	Click on a listing card	Item detail page loads with title, price, description, picture,	PASS	PASS

5. You also must perform the testing as per the plan above on 2 major WWW browsers of your choice and record the results in a form above (PASS or FAIL).

The QA test plan was executed using both Google Chrome and Microsoft Edge browsers. All major marketplace search and browsing functions worked with ease and without errors in both browsers. No crashes, glitches, broken links, display issues occurred. Search functionality, category filtering, and item listing navigation behaved as expected across both testing environments.

6. GenAI use: Did not use AI

4. Peer Code Review:

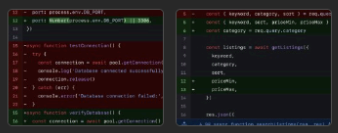
Sebastian Emilliano Cervantez

Reply | Reply all | Forward

To: Ryan Zhang; Abhay Bajaj; Kenny Camarra; Dylan Jon Alec Cordoni; Daniel De Jesus; Diego Fernando Rivera-Soto

Wed 5/20/2026 6:41 PM

You replied on Wed 5/20/2026 6:51 PM



2 attachments (886 KB) Save all to OneDrive - San Francisco State University Save All Attachments

Requesting code review implementation on backend files below.

- Fast fail start implementation
- searchController.js
- SearchModel.js

Sebastian Emiliano Cervantez

...

RZ

Ryan Zhang

Reply | Reply all | Forward

To: Sebastian Emilliano Cervantez

Wed 5/20/2026 6:51 PM

Hey Sebastian,

Overall, good work!

General themes of what I saw when I went over your code:

You had header comments at the top of the code, which is great. But they unfortunately didn't have any inline commenting, which would be really helpful when others are trying to read your code. I suggest that you include short comments about a function when you complete it, like what inputs, what it does, what it returns.

You do a good job of validating and sanitizing inputs, which is critical in catching errors. There was one issue where in the searchModel file, you used an embedded userId instead of the placeholder ?. This use can be prone to SQL injections, so I recommend that you use the '?' instead. Otherwise, you practiced good security practices.

RZ

Ryan Zhang

Reply | Reply all | Forward

To: Sebastian Emilliano Cervantez

Wed 5/20/2026 6:51 PM

Hey Sebastian,

Overall, good work!

General themes of what I saw when I went over your code:

You had header comments at the top of the code, which is great. But they unfortunately didn't have any inline commenting, which would be really helpful when others are trying to read your code. I suggest that you include short comments about a function when you complete it, like what inputs, what it does, what it returns.

You do a good job of validating and sanitizing inputs, which is critical in catching errors. There was one issue where in the searchModel file, you used an embedded userId instead of the placeholder ?. This use can be prone to SQL injections, so I recommend that you use the '?' instead. Otherwise, you practiced good security practices.

You also generally following our naming conventions, which is great. There was one instance where you used item instead of listing in the controller file, but otherwise everything is consistent.

There was also a bug where is session doesn't exist, the req.session.userId check will crash, so instead you should probably use session?.userId.

Otherwise, I think you did great work here. Just some fixes to the SQL input and adding more comments and I think we'll be great.

Thanks,
Ryan

...

Reply | Forward

application > backend > models > searchModel.js > normalizeCategories > raw

```
// Builds dynamic search query; only active approved listings unless owner views own item elsewhere.
// REVIEW: contains necessary basic header comment. could be more a bit more descriptive
const { pool } = require('../database')

// REVIEW: file does not contain in-line commenting.
// REVIEW: file follows naming conventions

// REVIEW: without comments a bit unclear what this function does at first glance
// REVIEW: good use of a helper function to sanitize category input
function normalizeCategories(category) {
  // REVIEW: general error/input handling -> good
  if (category === undefined || category === null || category === '') return []
  // REVIEW: nice array validation, ensuring that category will be an array and if it isn't converts to array
  const raw = Array.isArray(category) ? category : [category]
  return raw
    .flatMap((c) => String(c).split(','))
    .map((c) => c.trim())
    .filter(Boolean)
}

async function getListings({
  keyword = '',
  category,
  sort = '',
  priceMin = '',
  priceMax = '',
  userId = null
}) {
  // REVIEW: good use of left join, as users may not have any favorites
  // REVIEW: sql query gets listing_id, title, desc, price, category, a bool for isFavorited and isOwner
  // REVIEW: columns for all listings that are active and approved.
  // REVIEW: nice use of when then to create booleans
  // REVIEW: good practice to use ? placeholder instead of userId
  let sql = `
        SELECT
            l.listing_id,
            l.title,
            l.description,
            l.price,
            c.category_name,
            (
                SELECT li.image_url
                FROM listing_images li
                WHERE li.listing_id = l.listing_id
                LIMIT 1
            ) AS image_url,
            CASE
              WHEN uf.favorite_id IS NOT NULL THEN TRUE
```

```
            WHEN uf.favorite_id IS NOT NULL THEN TRUE
            ELSE FALSE
          END AS isFavorited,
          CASE
            WHEN l.user_id = ${userId} THEN TRUE
            ELSE FALSE
          END AS isOwner
        FROM listings l
        JOIN Categories c ON l.category = c.category_id
        LEFT JOIN user_favorites uf
          ON uf.listing_id = l.listing_id
          AND uf.user_id = ${userId}
        WHERE l.active = 1 AND l.approval_status = 'Approved'`

  const params = []
  // REVIEW: good use of SQL placeholders to prevent SQL injection here
  const kw = typeof keyword === 'string' ? keyword.trim() : ''
  if (kw) {
    sql += ' AND (l.title LIKE ? OR l.description LIKE ?)'
    params.push(`%${kw}%`, `%${kw}%`)
  }

  const categories = normalizeCategories(category)
  // REVIEW: code doesn't add filtering unless categories exists or is not empty good
  if (categories.length > 0) {
    const placeholders = categories.map(() => '?').join(', ')
    sql += ` AND LOWER(TRIM(c.category_name)) IN (${placeholders})`
    params.push(...categories.map((c) => c.toLowerCase()))
  }


  if (priceMin !== '' && priceMin !== undefined && priceMin !== null) {
    const n = Number(priceMin)
    if (!Number.isNaN(n)) {
      sql += ' AND l.price >= ?'
      params.push(n)
    }
  }
  if (priceMax !== '' && priceMax !== undefined && priceMax !== null) {
    const n = Number(priceMax)
    if (!Number.isNaN(n)) {
      sql += ' AND l.price <= ?'
      params.push(n)
    }
  }
```

```
  const categories = normalizeCategories(category)
  // REVIEW: code doesn't add filtering unless categories exists or is not empty good
  if (categories.length > 0) {
    const placeholders = categories.map(() => '?').join(', ')
    sql += ` AND LOWER(TRIM(c.category_name)) IN (${placeholders})`
    params.push(...categories.map((c) => c.toLowerCase()))
  }


  if (priceMin !== '' && priceMin !== undefined && priceMin !== null) {
    const n = Number(priceMin)
    if (!Number.isNaN(n)) {
      sql += ' AND l.price >= ?'
      params.push(n)
    }
  }
  if (priceMax !== '' && priceMax !== undefined && priceMax !== null) {
    const n = Number(priceMax)
    if (!Number.isNaN(n)) {
      sql += ' AND l.price <= ?'
      params.push(n)
    }
  }

  if (sort === 'price_low') {
    sql += ' ORDER BY l.price ASC'
  } else if (sort === 'price_high') {
    sql += ' ORDER BY l.price DESC'
  } else if (sort === 'newest') {
    sql += ' ORDER BY l.created_at DESC'
  } else {
    sql += ' ORDER BY l.created_at DESC'
  }

  const [rows] = await pool.query(sql, params)
  return rows
}
// REVIEW: code regularly checks for valid input and sanitizes inputs, good practice
module.exports = {
  getListings,
}
```

```
// Public search: keyword, category, price range, sort. Favorites flag if session exists.
// REVIEW: contains necessary basic header comment
const { getListings } = require('../models/searchModel')
// REVIEW: file missing inline commenting

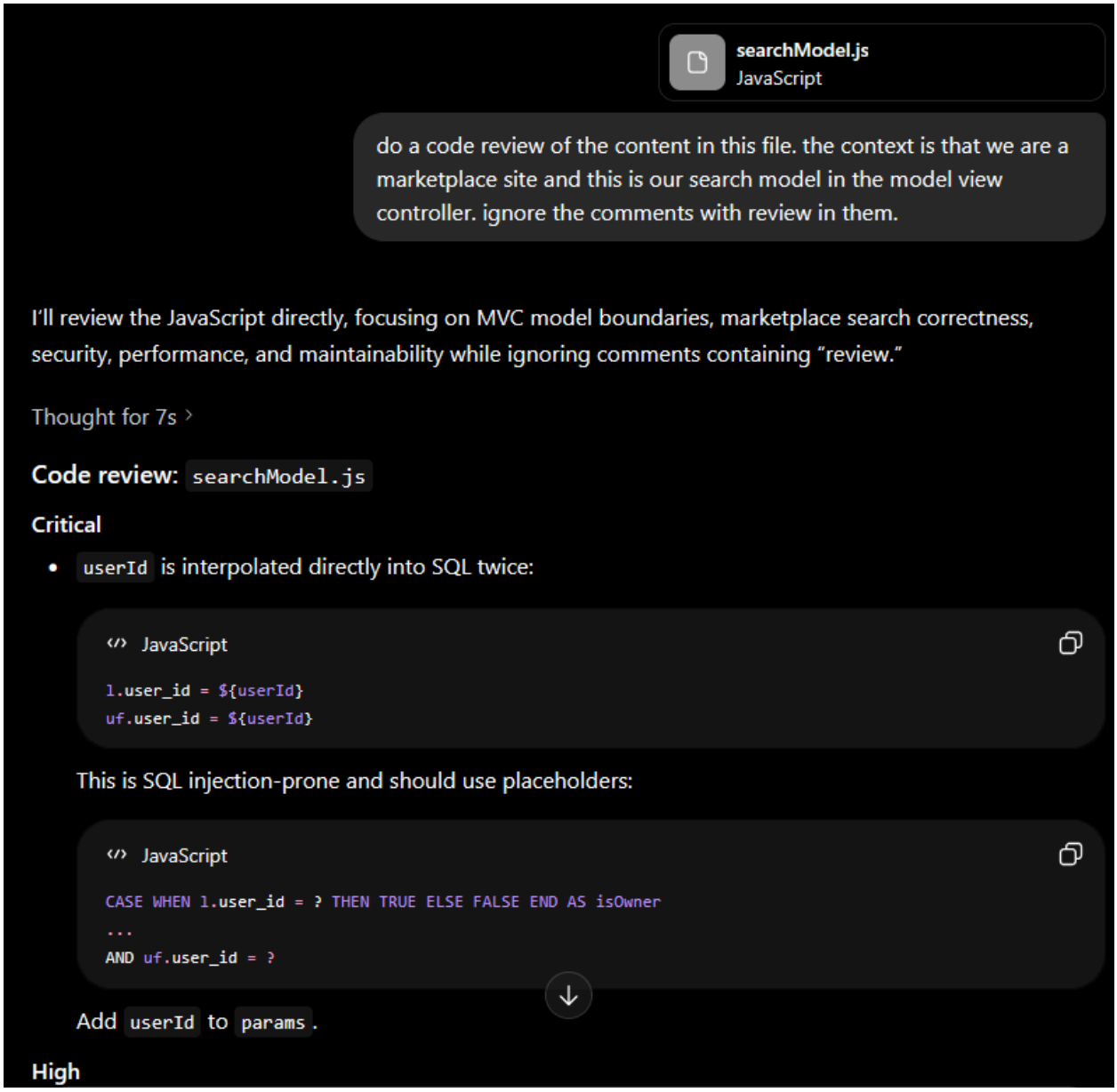
// REVIEW: generally follows naming conventions
// REVIEW: handles errors using try catch model
async function searchListings(req, res) {
  try {
    const { keyword, sort, priceMin, priceMax } = req.query
    const category = req.query.category
    // REVIEW: req.session.userId can crash if session is undefined
    const userId = req.session.userId || null

    const listings = await getListings({
      keyword,
      category,
      sort,
      priceMin,
      priceMax,
      userId
    })
    // REVIEW: should use listings here instead of items
    res.json({
      items: listings,
      total: listings.length,
    })
  } catch (err) {
    console.error(err)
    res.status(500).json({ error: err.message })
  }
}

module.exports = {
  searchListings,
}

```

GenAI code review session



- No pagination or result limit. Marketplace search can become slow and expensive as listings grow. Add `LIMIT` / `OFFSET`.
- `LIKE '%keyword%'` will not scale well. Consider full-text search for title/description.
- The image subquery has no ordering:

```SQL
LIMIT 1
```

  This returns an arbitrary image. Use something deterministic, such as `ORDER BY li.is_primary DESC, li.created_at ASC`.

Medium

- `LOWER(TRIM(c.category_name))` may prevent index usage. Prefer normalizing category names in the database or filtering by `category_id`.
- Price filters silently ignore invalid values. That may hide bad client input. Consider validating earlier and returning a clear error.
- No handling for `priceMin > priceMax`.
- Negative prices are accepted unless the database prevents them.
- `isFavorited` may duplicate listing rows if `user_favorites` allows multiple rows for the same user/listing. Add a unique constraint or use `EXISTS`.

Low

- Table name casing is inconsistent:

```SQL
JOIN Categories c
```

  If the rest of the schema uses lowercase names, this can break on case-sensitive systems.
- `TRUE` / `FALSE` may come back as `1` / `0` depending on the MySQL driver. Normalize in JS if the API

expects booleans.

- `sort = ''` defaults to newest, which is fine, but this could be simplified.

Suggested fix for the biggest issue

JavaScript

```
const params = [userId, userId]

let sql = `
  SELECT
    l.listing_id,
    l.title,
    l.description,
    l.price,
    c.category_name,
    (
      SELECT li.image_url
      FROM listing_images li
      WHERE li.listing_id = l.listing_id
      ORDER BY li.is_primary DESC, li.created_at ASC
      LIMIT 1
    ) AS image_url,
    CASE WHEN uf.favorite_id IS NOT NULL THEN TRUE ELSE FALSE END AS isFavorited,
    CASE WHEN l.user_id = ? THEN TRUE ELSE FALSE END AS isOwner
  FROM listings l
  JOIN Categories c ON l.category = c.category_id
  LEFT JOIN user_favorites uf
    ON uf.listing_id = l.listing_id
    AND uf.user_id = ?
  WHERE l.active = 1 AND l.approval_status = 'Approved'
`
```

Overall: the structure is reasonable for an MVC model, but the direct `userId` interpolation is the main blocker.

searchController.js
JavaScript

do a code review of this file as well, also ignoring REVIEW comments

Thought for a few seconds ›

Code review: `searchController.js`

High

- `req.session.userId` can throw if `req.session` is undefined:

```JavaScript
const userId = req.session?.userId || null
```

- The API exposes internal error messages:

```JavaScript
res.status(500).json({ error: err.message })
```

  Use a generic client response and log the real error server-side:

```JavaScript
res.status(500).json({ error: 'Search failed' })
```

Medium

- Query params are passed through without validation or normalization. Validate `priceMin`, `priceMax`, `sort`, and `category` before calling the model.
- `total: listings.length` only reports the numb[illegible]returned, not the total matching count. This becomes misleading once pagination is added.
- No pagination params are accepted. Marketplace search should support `limit` and `offset` or `page`.

Low

- `category` is destructured separately from the other query params for no clear reason:

```javascript
const { keyword, sort, priceMin, priceMax, category } = req.query
```

- `console.error(err)` is acceptable for simple apps, but a structured logger is better in production.

Suggested revision

```javascript
async function searchListings(req, res) {
  try {
    const {
      keyword,
      category,
      sort,
      priceMin,
      priceMax,
      limit = 20,
      offset = 0,
    } = req.query

    const userId = req.session?.userId || null

    const listings = await getListings({
      keyword: keyword?.trim(),
      category,
      sort,
      priceMin,
      priceMax,
      limit: Number(limit),
      offset: Number(offset),
      userId,
    })

    res.json({
      items: listings,
      total: listings.length,
    })
  } catch (err) {
    console.error('Search listings failed:', err)
    res.status(500).json({ error: 'Search failed' })
  }
}
```

Main issue: this controller trusts query params too much and leaks server errors to the client.

We used ChatGPT Edu, the ChatGPT provided by SFSU.

We prompted it to do a code review on the contents of searchModel.js and searchController.js, the backend implementation of our search function. We also used it to review certain SQL syntax. It was helpful as a look into what modern safe coding practices are, since as students, we may not be the most familiar with what safety practices real developers use in the software, as we're all still learning. This also gave us quick insights into what could possibly be vulnerabilities or bugs in our programs and how to address them if they exist.

For example, we prompted it with "do a code review of the content in this file. the context is that we are a marketplace site and this is our search model in the model view controller. ignore the comments with review in them. " and attached the file. ChatGPT told us that the use of ${userId} was dangerous because it didn't protect against SQL injection, which was an area of safety concern we went over in class.

Another example is after the GenAI code review, we prompted it with "explain the danger of sql injection and the use of embedded userId more", asking it to go into further detail. It explained that even though it looks harmless, if someone could influence userId, they could change the SQL query. It gave an example of if userId was 5 OR 1=1, it could return all the users in the table.

Utility of GenAI: Medium

5. Self-check on best practices for security – ½ page

Asset to be Protected	Types of Possible/Expected Attacks	Consequence of Security Breach	Strategy to Mitigate/Protect the Asset
User Password	Credential theft, DB leak, brute force	Account takeover, users reuse password on other sites leading to more harm	Passwords must never be stored in plain text. Use Salted hashing before saving to the DB. Enforce strong password rules.
User Registration Data (email, profile info)	Data exposure, code injection from fields, account enumeration	privacy breach, unauthorized access, harm to reputation	Validate and sanitize inputs, enforce SFSU domain, avoid error messages that give away unnecessary information.
Database	SQL injection,	Data corruption &	Use parameterized queries,

	unauthorized DB access, stolen credentials	deletion, data leak, service outages	principle of least-privilege, no DB credentials in source code, restrict DB network access.
Endpoints	Insecure endpoints, bad JSON payloads, CORS misuse	Unauthorized cross-site calls, crashing, unexpected behavior in page serving	Validate request schemes, handle errors without giving information to the end-user, keep dependencies updated, keep CORS restricted to a known origin.
Credentials	Leakage of sensitive/useful data/log-in information from Git history	Compromised Database access and all contents.	Can store credentials in a .env locally and make sure to never commit real credentials, can rotate/change credentials if needed, use different credentials for the dev & production versions.

Confirmation of PW encryption in the DB: **SOMEONE NEEDS TO DO THIS**

We confirm that the passwords in our database were encrypted using bcrypt.

Confirmation of Input data validation (what is being validated and what code you used):

What We Validate:	**Code Used To Validate:**
Email used in Registration	const emailValid = form.email.endsWith('@sfsu.edu')
Full Name	const fullNameValid = form.fullName.trim().length > 0
Username	const usernameValid = /^[A-Za-z]{5,}$/.test(form.username
Password	const passwordValid = form.password.length >= 8 && /[0-9]/.test(form.password) && /[!@#$%^&*]/.test(form.password)
Password Confirmation Match	const passwordsMatch = form.confirmPassword.length > 0 && form.password === form.confirmPassword
SearchInput is 40 Characters	<input className="gb-search-input" type="text" placeholder={placeholder} value={query} maxLength={40} onChange={(e) => onQueryChange(e.target.value)} aria-label={searchInputAriaLabel} autoFocus={autoFocus} />

Terms&Conditions are accepted	<input type="checkbox" name="agreedToTerms" checked={form.agreedToTerms} onChange={handleChange} />

6. Self-check of the adherence to original Non-functional specs

1. DONE - Application shall be developed, tested and deployed using tools and cloud servers approved by Class CTO and as agreed in M0
2. DONE - Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers
3. ISSUE All our selected application functions shall be rendered well on mobile devices (no native app to be developed)

   Unfortunately, we did not have the time to make the site render on well on mobile device as our time was spent making the site functional and implementing P1 functionality.
4. DONE Posting sales information and messaging to sellers shall be limited only to SFSU students
5. DONE Critical data shall be stored in the database on the team's deployment server.
6. DONE No more than 50 concurrent users shall be accessing the application at any time
7. DONE Privacy of users shall be protected
8. DONE The language used shall be English (no localization needed)
9. DONE Application shall be very easy to use and intuitive
10. DONE Application shall follow established architecture patterns
11. DONE Application code and its repository shall be easy to inspect and maintain
12. DONE Google analytics shall be used
13. DONE No e-mail clients or chat services shall be allowed. Interested users can only message to sellers via in-site messaging. One round of messaging (from user to seller) is enough for this application
14. DONE Pay functionality, if any (e.g. paying for goods and services) shall not be implemented nor simulated in UI.
15. DONE Site security: basic best practices shall be applied (as covered in the class) for main data items
16. DONE Media formats shall be standard as used in the market today
17. DONE Modern SE processes and tools shall be used as specified in the class, including collaborative and continuous SW development and GenAI tools
18. DONE The application UI (WWW and mobile) shall prominently display the following exact text on all pages "SFSU Software Engineering Project CSC 648-848, Spring 2026.

For Demonstration Only” at the top of the WWW page Nav bar. (Important to not confuse this with a real application). You must use this exact text without any editing.

3. Product Screenshots

Banner and Search and Nav Bar (Included on every page)



Home page with Footer (Footer present on every page)

Clothes | Text Books | Furniture | Electronics | Class Materials | Room Search

Recently Added



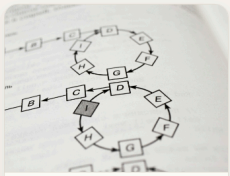
Old Chair
Brand new chair.
$99.00



Summer Studio Sublease Close to Campus
Furnished studio sublease available for the summer semester. Utilities and...
$400.00



Private Room Near SFSU Library
Quiet furnished private room within walking distance of SFSU. Includes WiFi...
$950.00



Algorithm Design Manual
Reference book for algorithms and coding competitions.
$30.00



JavaScript: The Good Parts
Classic programming book for frontend developers.
$12.00

Recently Viewed



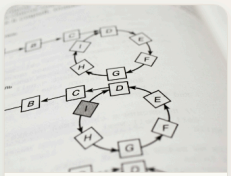
Old Chair
Brand new chair.
$99.00



Summer Studio Sublease Close to Campus
Furnished studio sublease available for the summer semester. Utilities and...
$400.00



Private Room Near SFSU Library
Quiet furnished private room within walking distance of SFSU. Includes WiFi...
$950.00



Algorithm Design Manual
Reference book for algorithms and coding competitions.
$30.00

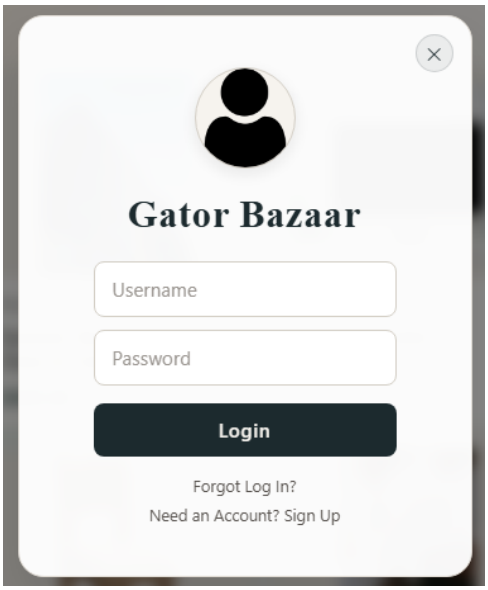


JavaScript: The Good Parts
Classic programming book for frontend developers.
$12.00

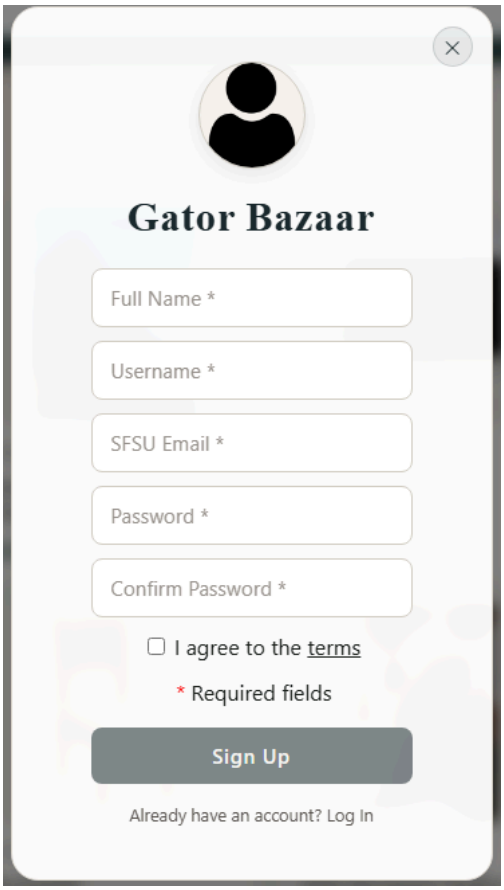
© 2026 GatorBazaar. All rights reserved.

Team Members

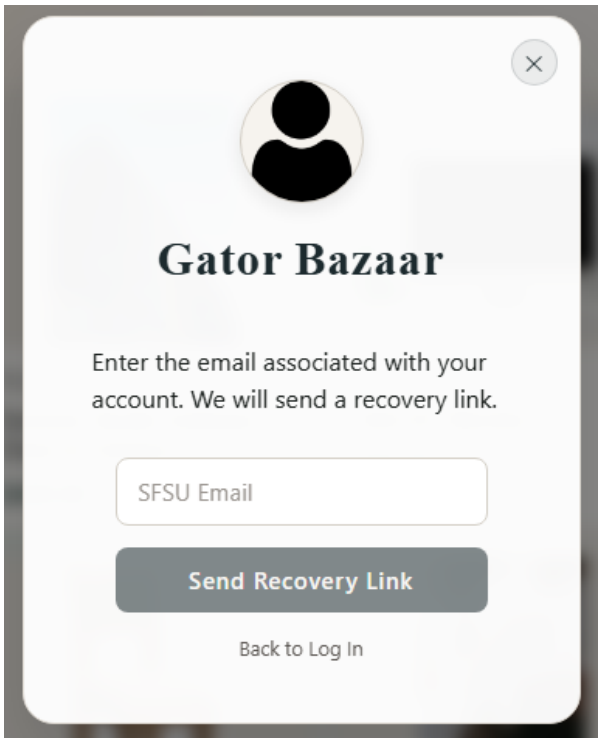
Login Modal



Registration Modal



Account Recover Modal



Dashboard Page

Seller Dashboard

Track your active listings and recently sold listings in one place.

Pending Listings

1 LISTINGS



Bookshelf
$85.00

Rejected Listings

2 LISTINGS



Banana
$900.00



Fish
$1200.00

Currently Listed

2 LISTINGS



Notebooks
$30.00



Old Chair
$99.00

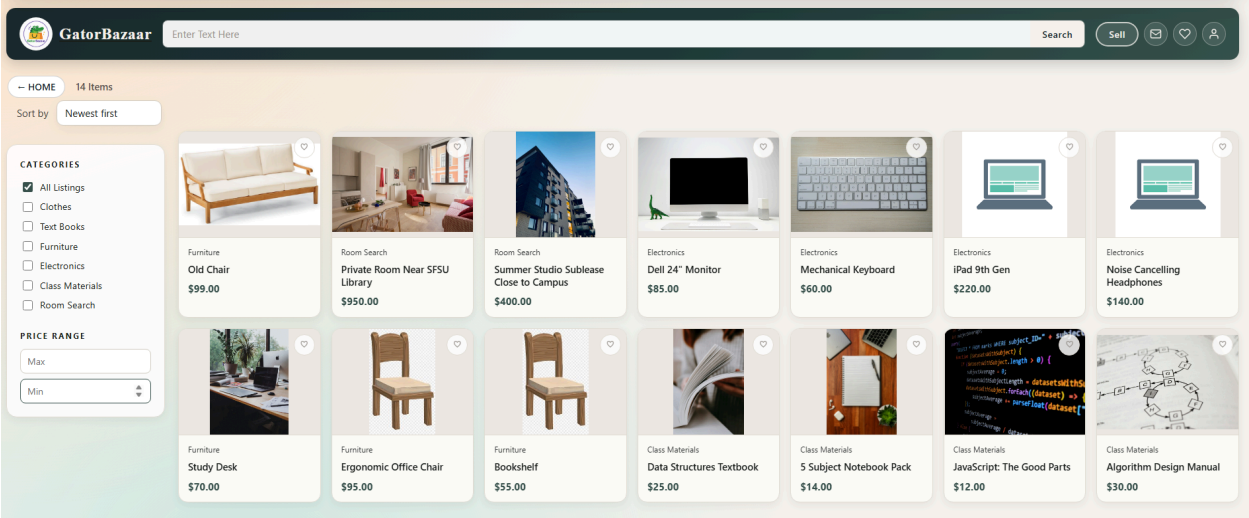
Sold listings

1 LISTINGS

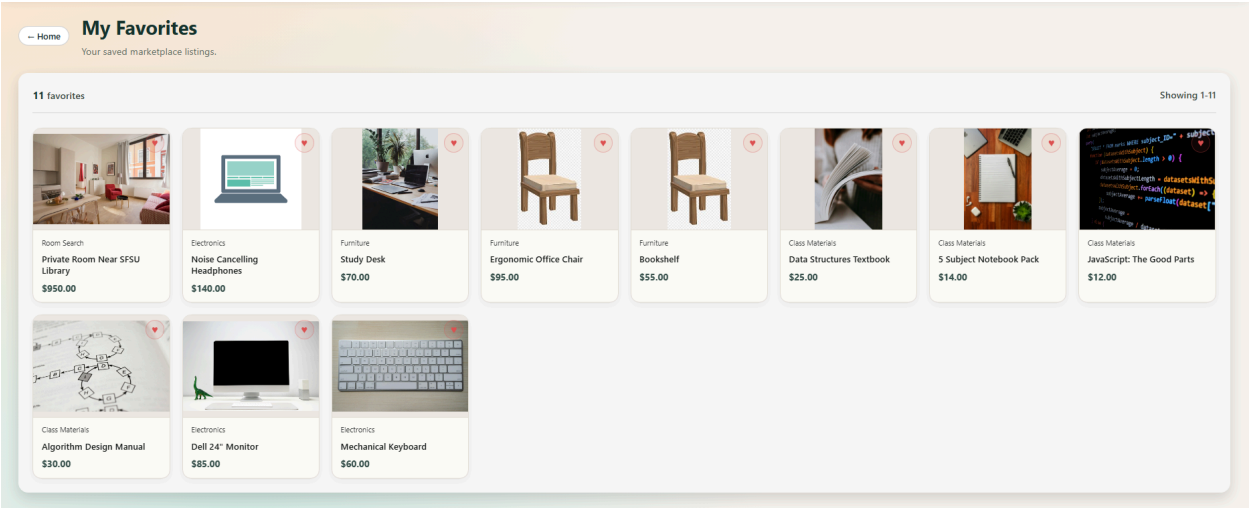


15-pack Pencils
Listed: $15.00
Sold: $0.00

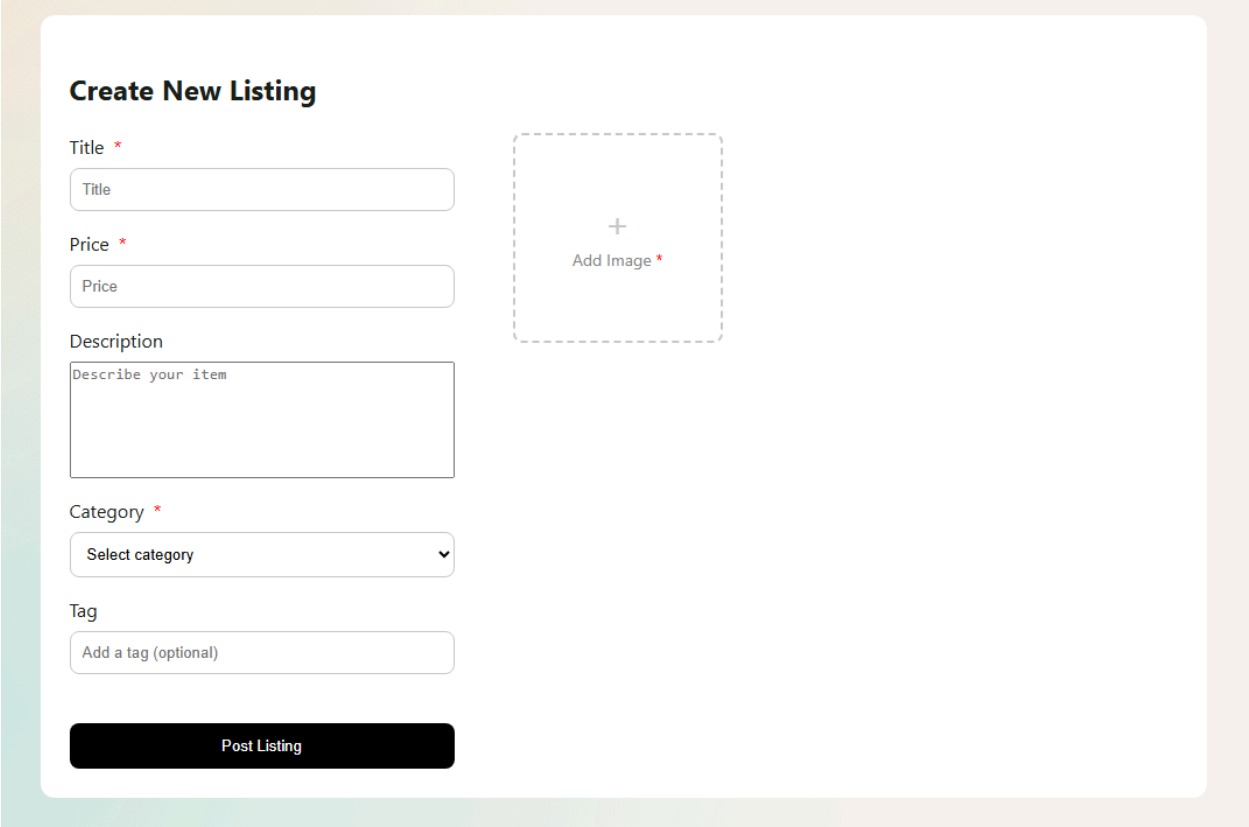
Search Page with Search and Nav Bar



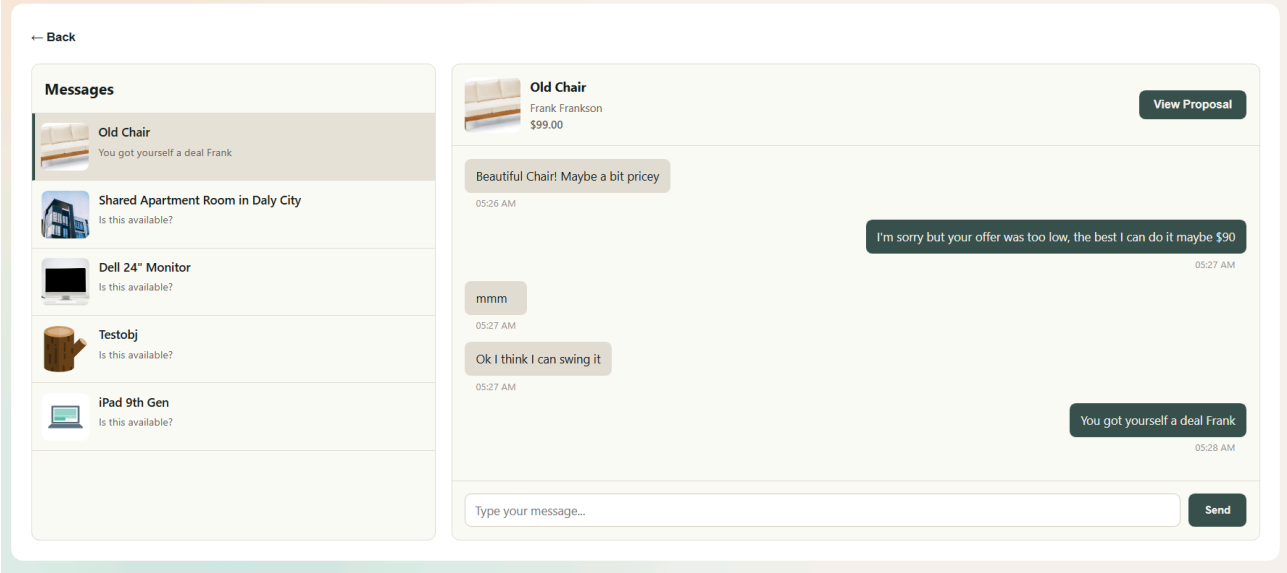
Favorites Page



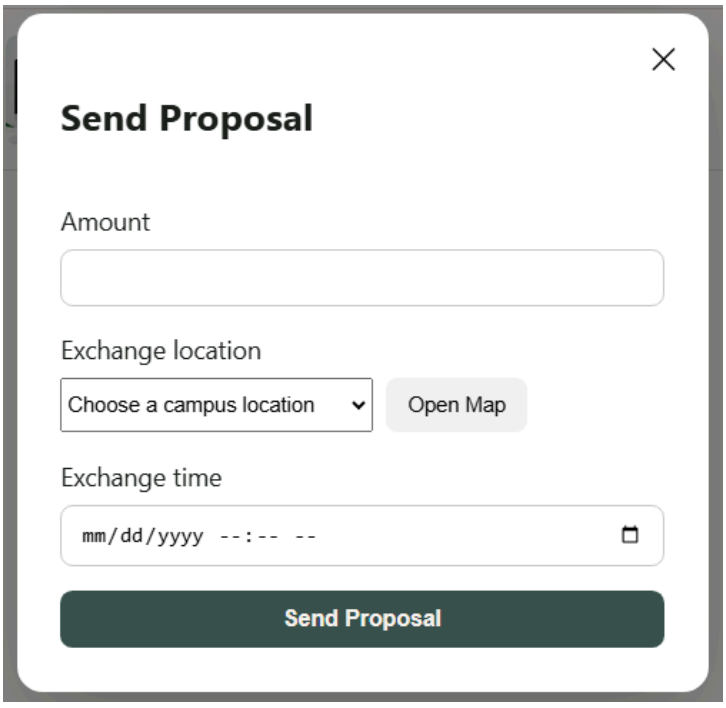
Create Listing Page



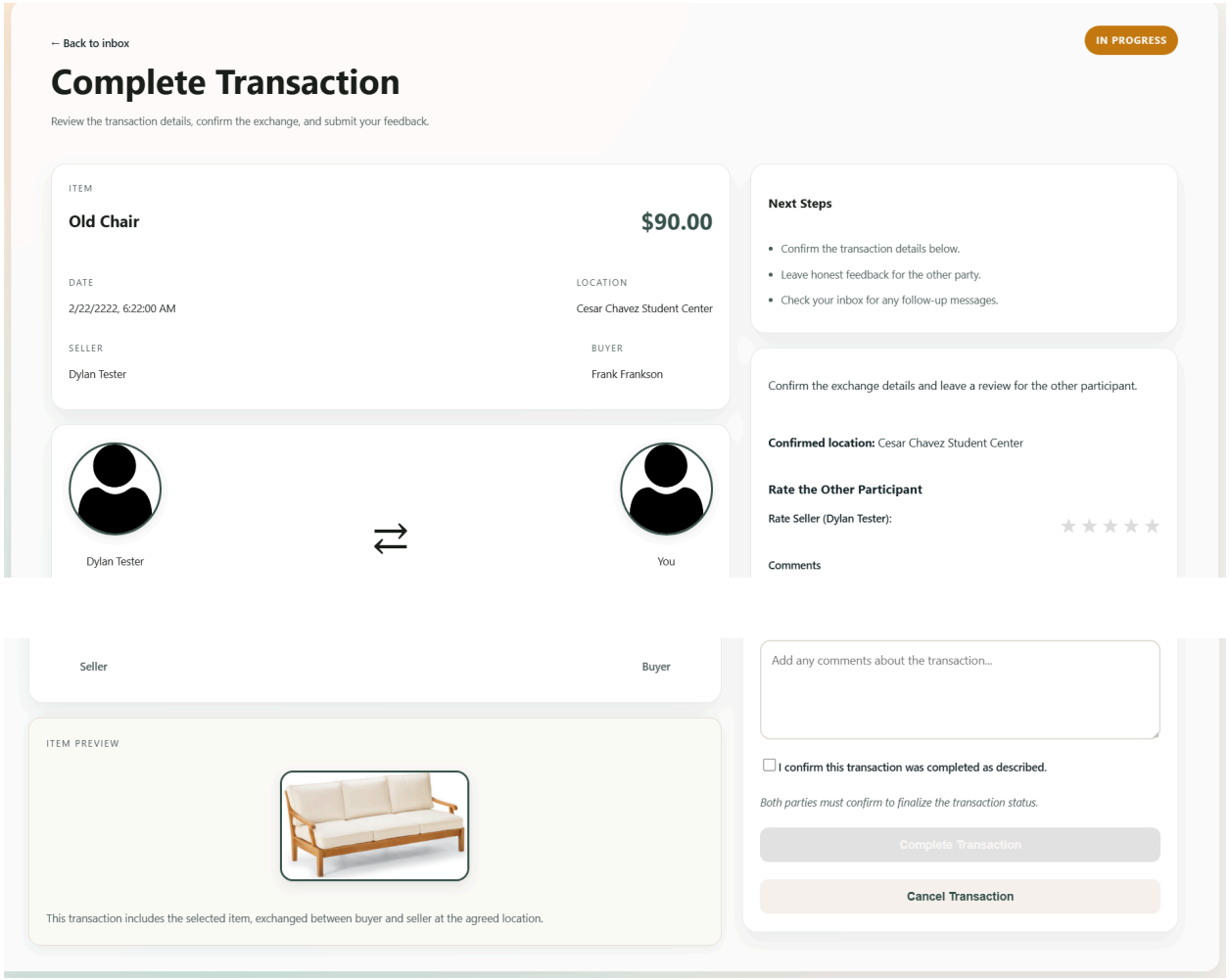
Inbox Page



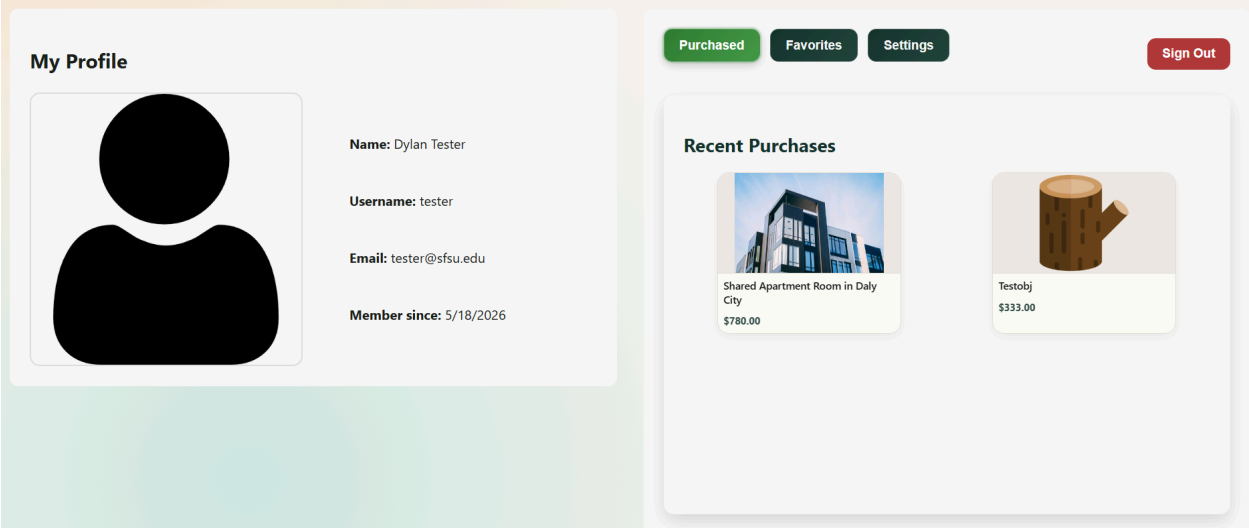
Proposal Modal



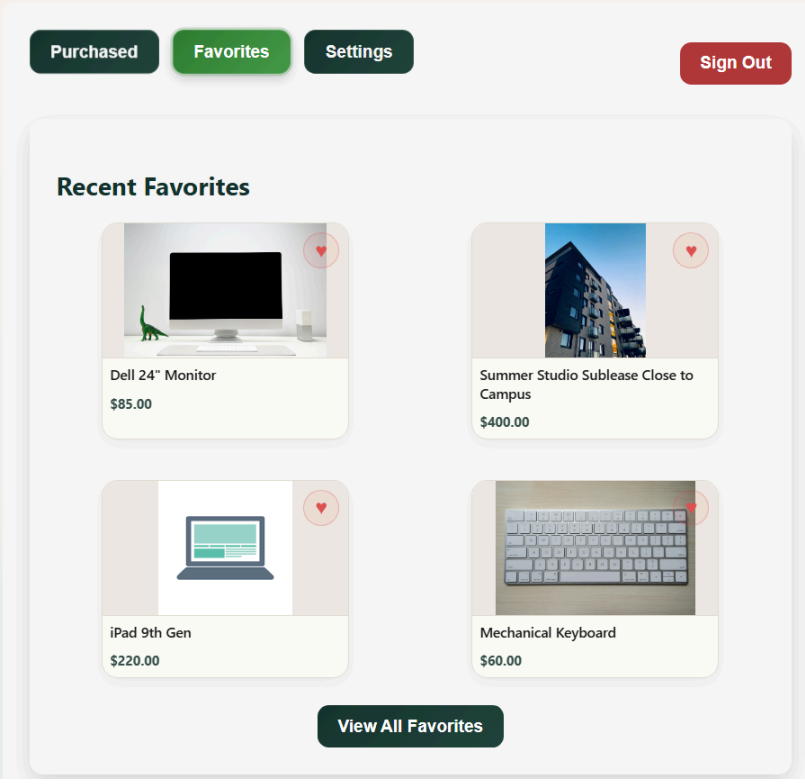
Complete Transaction Page



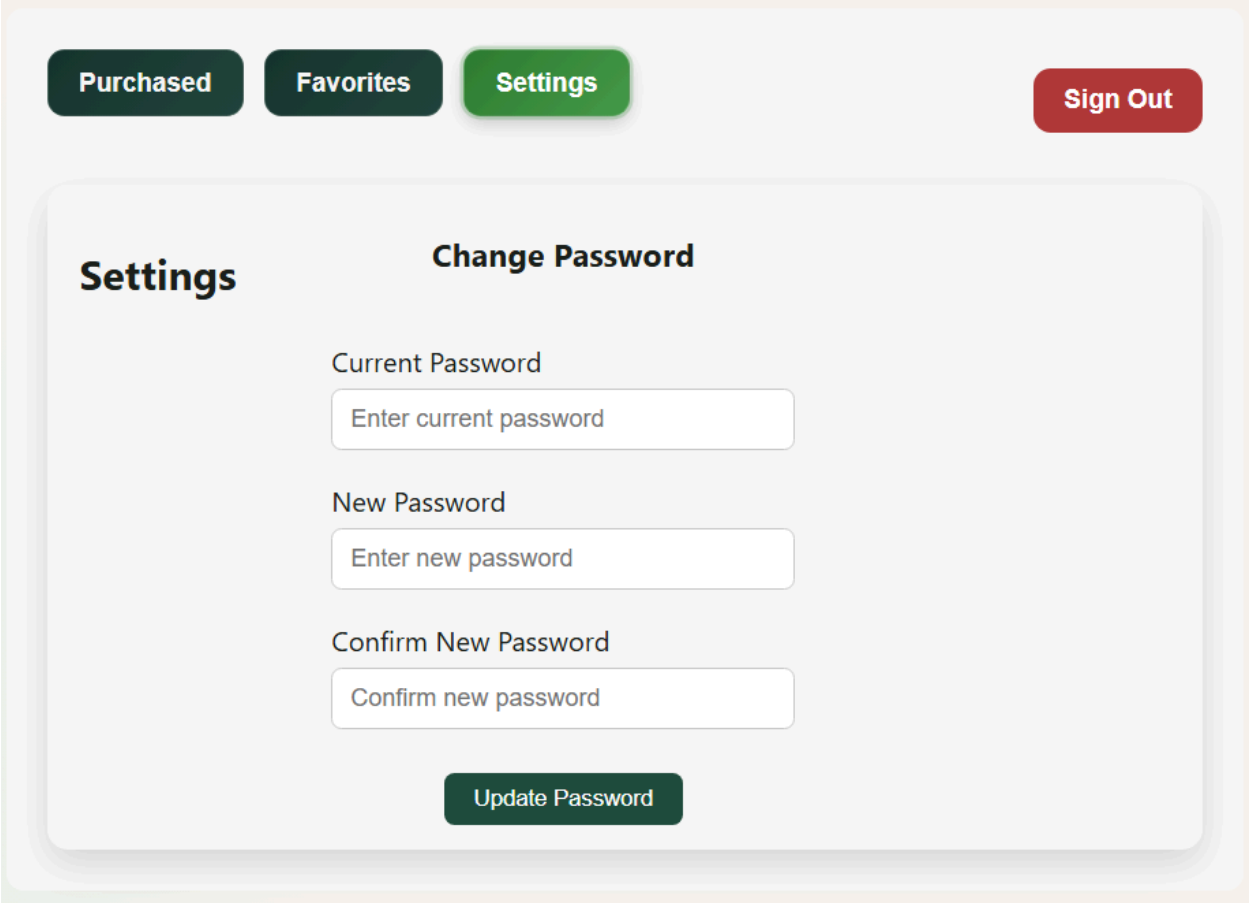
Account Page (Bought Listings tab)



Account Page (Favorites tab)



Account Page (Settings Tab)



4. Database Organization:

Software stack:

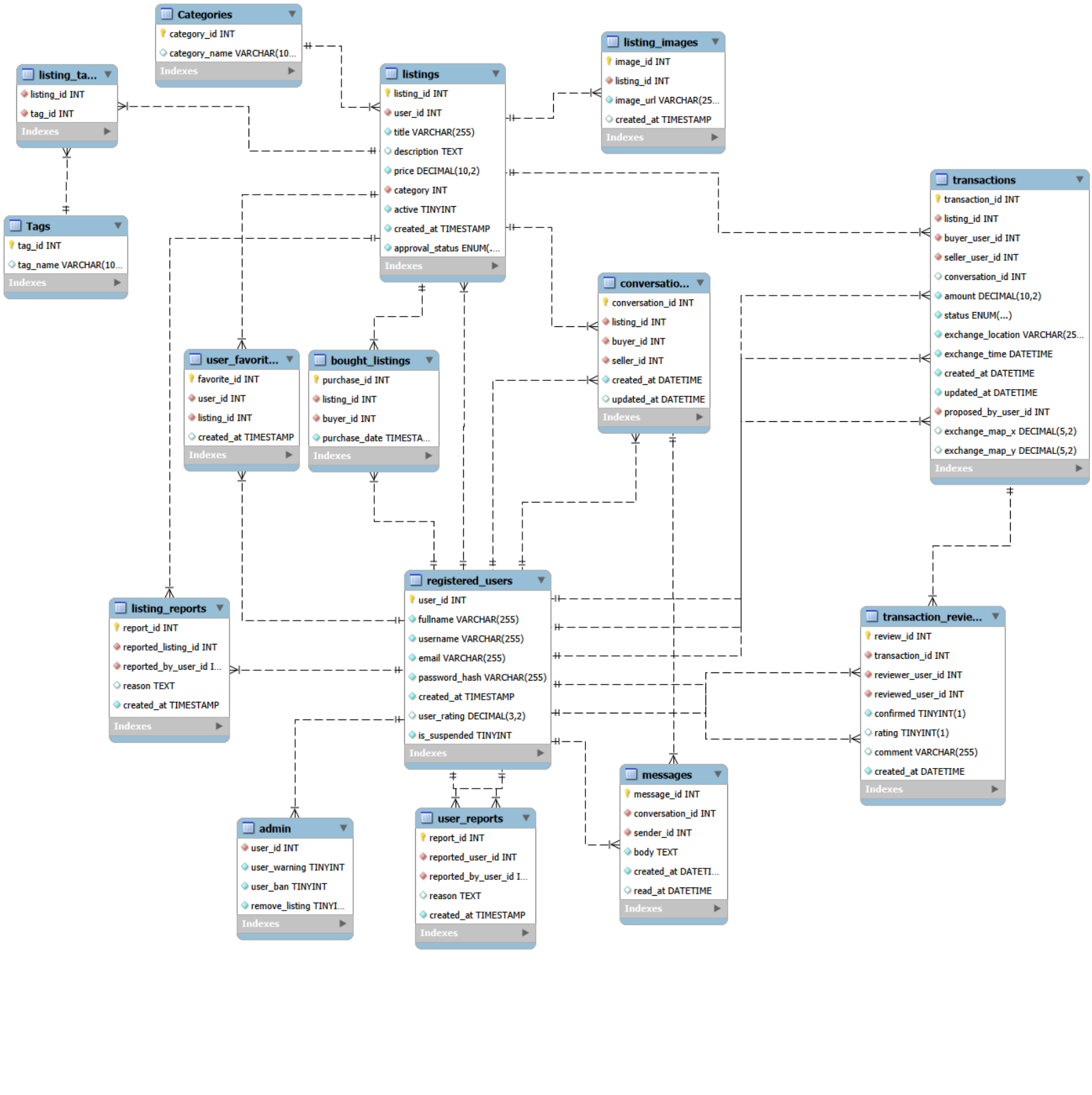
Server Host: AWS EC2 t3.small
Operating System: Ubuntu 24.04.1
Database: MySQL v 8.0.45
Web Server: Apache 2.4.58
Server-Side Language: JavaScript

Our application uses a single MySQL database named app_db hosted on the team's EC2 server. The schema supports student accounts, marketplace listings, categories, images, favorites, and buyer–seller messaging. The registered_userstable stores San Francisco State University student accounts using @sfsu.edu emails, while Categories defines listing types such as textbooks and room search. The listings and listing_images tables manage items for sale and their photos, user_favorites tracks saved listings, and conversations and messages handle buyer & seller chat threads and individual messages between users Key database Tables that were a main focus of our entire backend throughout our development consisted of registered users, Conversations, Messages, along with Categories

```
mysql> DESCRIBE listings;
+-------------+--------------+------+-----+-------------------+-------------------+
| Field       | Type         | Null | Key | Default           | Extra             |
+-------------+--------------+------+-----+-------------------+-------------------+
| listing_id  | int unsigned | NO   | PRI | NULL              | auto_increment    |
| user_id     | int unsigned | NO   | MUL | NULL              |                   |
| title       | varchar(255) | NO   |     | NULL              |                   |
| description | text         | YES  |     | NULL              |                   |
| price       | decimal(5,2) | NO   |     | NULL              |                   |
| category    | int          | NO   | MUL | NULL              |                   |
| active      | tinyint      | NO   |     | 1                 |                   |
| created_at  | timestamp    | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-------------+--------------+------+-----+-------------------+-------------------+
8 rows in set (0.00 sec)
```

```
mysql> DESCRIBE registered_users;
+--------------+---------------------+------+-----+-------------------+-------------------+
| Field        | Type                | Null | Key | Default           | Extra             |
+--------------+---------------------+------+-----+-------------------+-------------------+
| user_id      | int unsigned        | NO   | PRI | NULL              | auto_increment    |
| fullname     | varchar(255)        | NO   |     | NULL              |                   |
| username     | varchar(255)        | NO   |     | NULL              |                   |
| email        | varchar(255)        | NO   | UNI | NULL              |                   |
| password_hash | varchar(255)       | NO   |     | NULL              |                   |
| created_at   | timestamp           | NO   |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| user_rating  | decimal(3,2) unsigned | YES |     | 0.00              |                   |
| is_suspended | tinyint             | NO   |     | 0                 |                   |
+--------------+---------------------+------+-----+-------------------+-------------------+
8 rows in set (0.00 sec)
```

```
mysql> DESCRIBE Categories;
+---------------+--------------+------+-----+---------+----------------+
| Field         | Type         | Null | Key | Default | Extra          |
+---------------+--------------+------+-----+---------+----------------+
| category_id   | int          | NO   | PRI | NULL    | auto_increment |
```



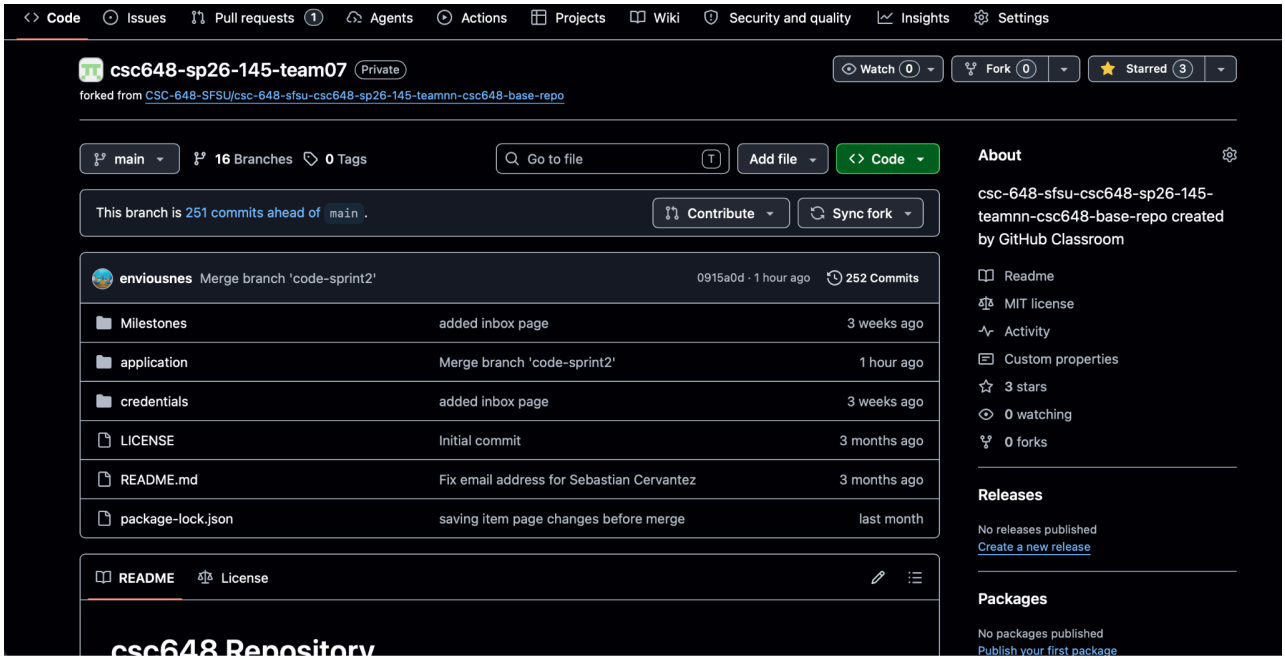
5. GitHub organization:

The primary branch used for the project was the **main** branch. In addition, the repository contained multiple development and feature branches created by team members for implementing and testing features before merging into main. At the time of submission, the repository contained 16 branches total. Our Main branches for organization throughout development consisted as follows.

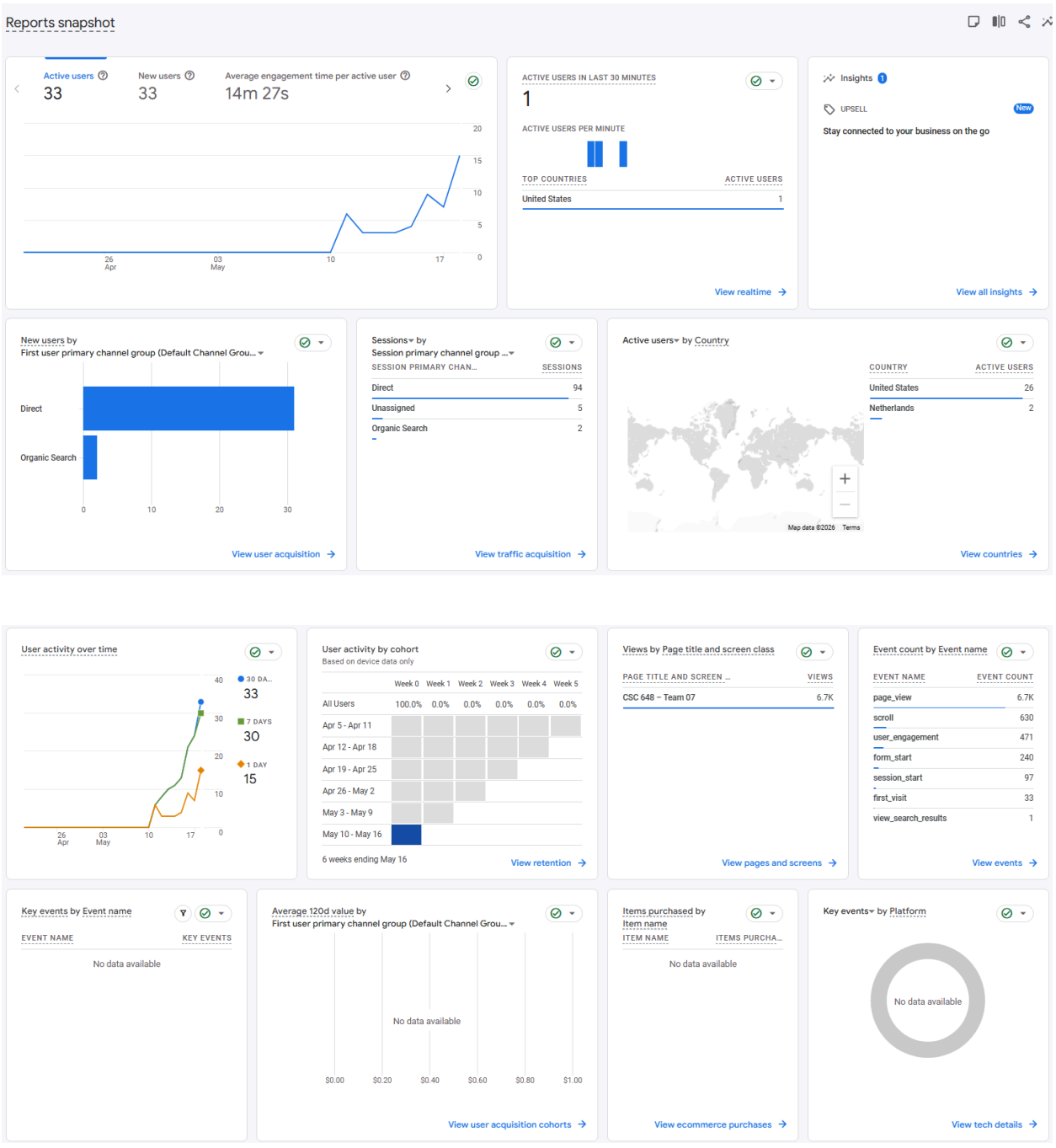
- Front-end-CSS
- fix/account-page
- Admin-Approval-Feature
- Code-sprint2
- fix/cors
- fix/basicSecuritySearchBarRegistration
- Feature-DeleteListing
- fix/homepage-listings
- Messages-Feature-Backend
- Favorite
- fix/StyleOfDropRatingHome
- feature/favorites

Access to the main branch was restricted to authorized team members within the GitHub organization. Team members primarily worked on separate feature branches and merged changes into main after testing and review to maintain project stability.

Below is a Screenshot of our Github homepage:

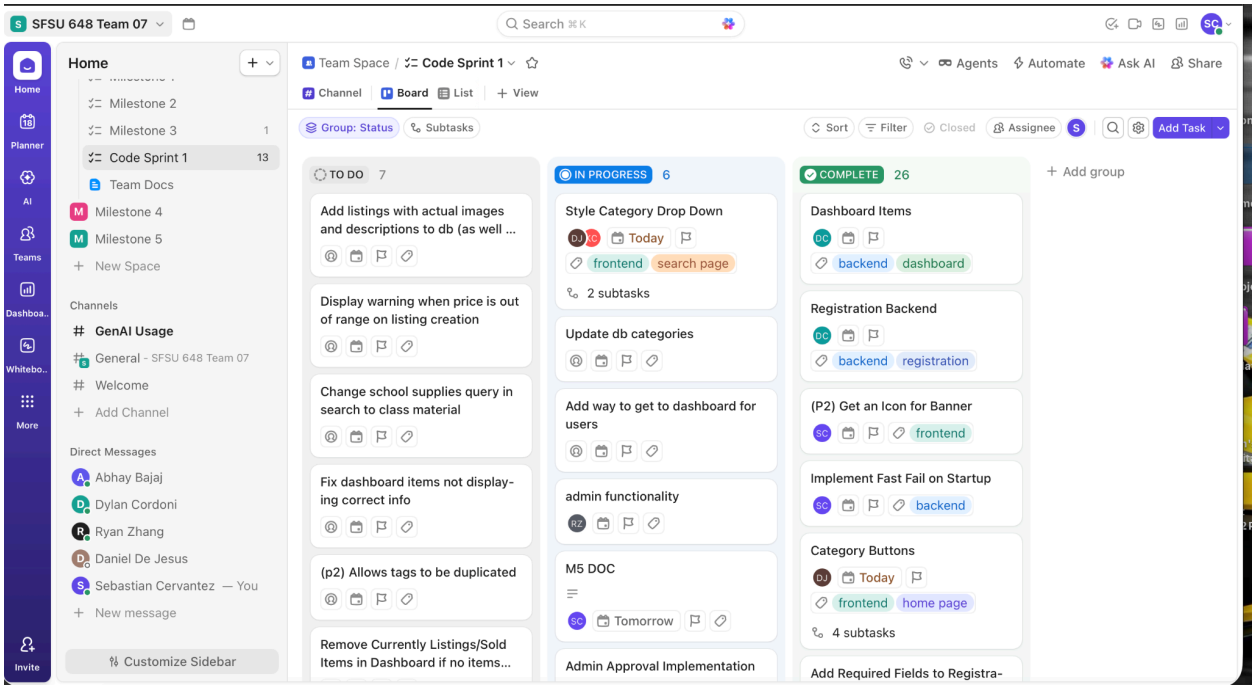

Code
Issues
Pull requests 1
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
csc648-sp26-145-team07
Private
forked from CSC-648-SFSU/csc-648-sfsu-csc648-sp26-145-teamnn-csc648-base-repo
Watch 0
Fork 0
Starred 3
main
16 Branches
0 Tags
Go to file
T
Add file
Code
This branch is 251 commits ahead of main .
Contribute
Sync fork
enviousnes Merge branch 'code-sprint2'
0915a0d · 1 hour ago
252 Commits
Milestones added inbox page 3 weeks ago
application Merge branch 'code-sprint2' 1 hour ago
credentials added inbox page 3 weeks ago
LICENSE Initial commit 3 months ago
README.md Fix email address for Sebastian Cervantez 3 months ago
package-lock.json saving item page changes before merge last month
README
License
csc648 Repository
About
csc-648-sfsu-csc648-sp26-145-teamnn-csc648-base-repo created by GitHub Classroom
Readme
MIT license
Activity
Custom properties
3 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package

6. Google analytics stats plot for your WWW site

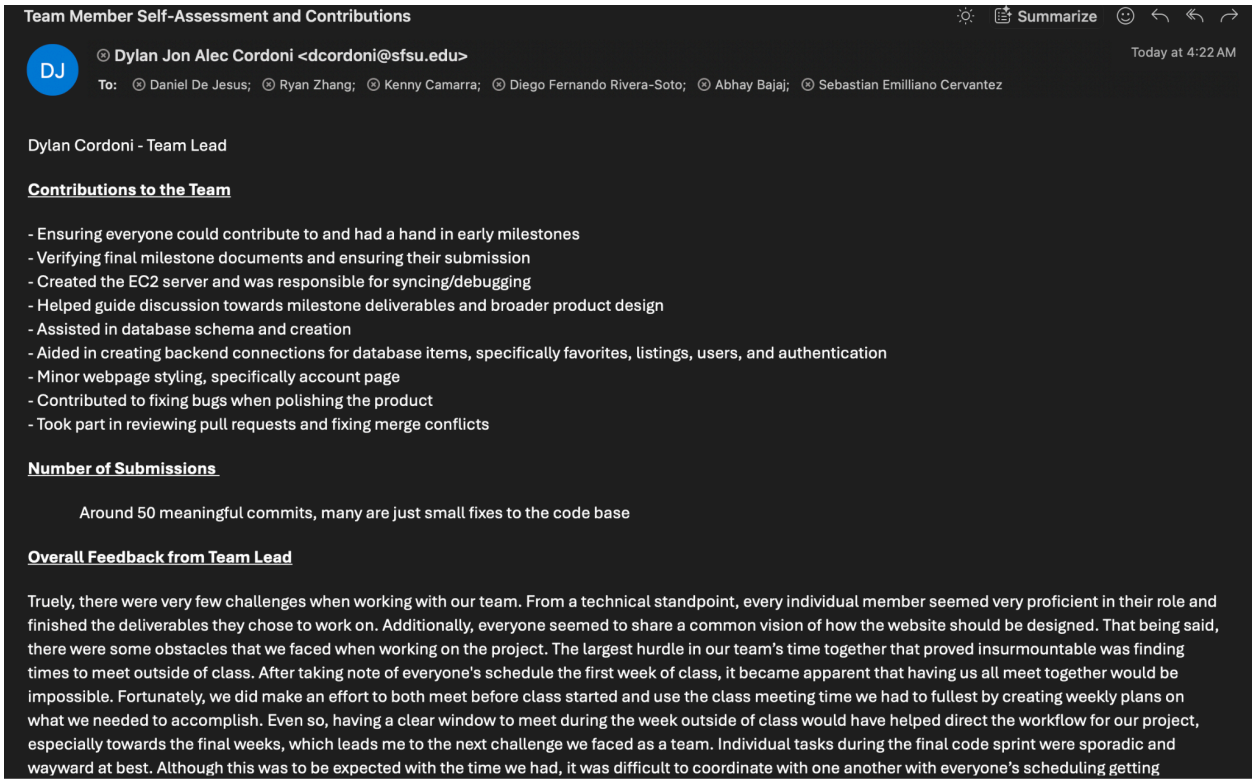


7. Project management:

Tools we used for tracking work from all team members was Clickup. This application was extremely efficient and kept all developers on track with due dates and plenty of work to go around to divide it all equally. Clickup and discord also helped with lots of communication when new tasks were put up for the respective backend or frontend developers to assign tasks to in progress or completed. Below is a screenshot of one of more recent boards of ToDo tasks along with completed and in progress tasks.



8. Team member self-assessment and contributions



Team Member Self-Assessment and Contributions

Summarize

DJ

Dylan Jon Alec Cordoni <dcordoni@sfsu.edu>

Today at 4:22 AM

To: Daniel De Jesus; Ryan Zhang; Kenny Camarra; Diego Fernando Rivera-Soto; Abhay Bajaj; Sebastian Emilliano Cervantez

Dylan Cordoni - Team Lead

Contributions to the Team

- Ensuring everyone could contribute to and had a hand in early milestones
- Verifying final milestone documents and ensuring their submission
- Created the EC2 server and was responsible for syncing/debugging
- Helped guide discussion towards milestone deliverables and broader product design
- Assisted in database schema and creation
- Aided in creating backend connections for database items, specifically favorites, listings, users, and authentication
- Minor webpage styling, specifically account page
- Contributed to fixing bugs when polishing the product
- Took part in reviewing pull requests and fixing merge conflicts

Number of Submissions

Around 50 meaningful commits, many are just small fixes to the code base

Overall Feedback from Team Lead

Truely, there were very few challenges when working with our team. From a technical standpoint, every individual member seemed very proficient in their role and finished the deliverables they chose to work on. Additionally, everyone seemed to share a common vision of how the website should be designed. That being said, there were some obstacles that we faced when working on the project. The largest hurdle in our team's time together that proved insurmountable was finding times to meet outside of class. After taking note of everyone's schedule the first week of class, it became apparent that having us all meet together would be impossible. Fortunately, we did make an effort to both meet before class started and use the class meeting time we had to fullest by creating weekly plans on what we needed to accomplish. Even so, having a clear window to meet during the week outside of class would have helped direct the workflow for our project, especially towards the final weeks, which leads me to the next challenge we faced as a team. Individual tasks during the final code sprint were sporadic and wayward at best. Although this was to be expected with the time we had, it was difficult to coordinate with one another with everyone's scheduling getting

weeks, which leads me to the next challenge we faced as a team. Individual tasks during the final code sprint were sporadic and wayward at best. Although this was to be expected with the time we had, it was difficult to coordinate with one another with everyone's scheduling getting increasingly more busy due to finals week. All in all, everyone performed very well individualistically as well as in the team setting despite these barriers that we faced.

As for GenAI usage, it was scarcely discussed within the team. However, we did explicitly use it for generating our site name, GatorBazaar. It's nearly impossible to google something without Gemini offering a potential solution, so it would be naive to think GenAI did not have a role in helping with our project. That being said, uses of other GenAI models such as Claude and ChatGPT were not discussed within the team meetings.

If I could go back to the beginning of the semester, there are certainly a few things I would do better. Firstly, I would take advantage of our task management tool more. Although we used Clickup early on in our project to keep track of milestone deliverables, it was not until the last couple of weeks that each Clickup task was focused on a very particular aspect of the site that needed to get done. This could have helped during our last month of development to outline exactly what we needed at each stage. I should have taken more time to write clear and specific actions for these Clickup tasks. Secondly, I would have had the team get a jump start on the coding aspect earlier than we did. With schedules already stacked, getting particular features out of the way when our Milestone 2 Vertical Prototype was due would have saved us a lot of time during our code sprint. There were many weeks where we knew our deliverables were few and that would have been the perfect time to add a little functionality to our site. Lastly, I would have tried to open my schedule up more for the team. If I would have taken an extra day off from work, I could have conferred with my team about tasks they were doing, get progress reports, and better encourage team discourse.

Other Notes for Instructor

Thanks for giving me such a good team.

Re: Team Member Self-Assessment and Contributions



Sebastian Emilliano Cervantez <scervantez@mail.sfsu.edu>

Today at 11:09 PM

To: Dylan Jon Alec Cordoni; Daniel De Jesus; Ryan Zhang; Kenny Camarra; Diego Fernando Rivera-Soto; Abhay Bajaj

Sebastian Cervantez Backend Lead

Contributions to the Team -

- Helped deploy server and set up all SQL implementations from tables of the database etc.
- Helped complete all documentation from M1-M5 for example Executive summaries, Product summaries.
- Created our database organization
- Helped in database schema/ creation with Ryan and Dylan
- Created initial dashboard when all members were needed for the frontend along with other CSS HTML implementation
- Created the Icon for the website we implemented for final use.
- Assisted in fixing bugs etc.
- Helped create the messages feature between users
- Also assisted in keeping team on track with design choices and processes with the TL

Number of Submissions

- Around 28 submissions whether it was fixes or new features. Verified through pull requests I opened.

This number I would say is low due to the fact I was working on many other contributions to the team. I found myself struggling a bit building a backend from the ground up. But thankfully for having great help from Dylan and Ryan they were able to help me learn many things and assigned plenty for me to complete. Verified through clickup.

Main challenges I encountered in this team project was understanding how to coordinate with this many people on this scale of a project. We had to make sure when we did meet, we knew what will be done and how it will get done. Another challenge I faced was relearning a lot of backend commands and processes. It had been awhile to catch me up to speed but I was able to do lots of research and come up with habits that kept my knowledge up to the standards it needed to be throughout the semester to stay ready and contribute throughout this whole project.

My experience with GenAI helped in times when I didn't understand how certain functions or processes worked. But as a team I recall using it to create multiple name iterations and icon iterations to decide on what we wanted best. We aren't graphic designers, so this helped to get a bit creative.

Overall, on what to do better based off what I was able to learn through SW slides and lectures. Understanding MVC and all other concepts like react more for example through UML etc. Just not be able to not only know why the team as a group is making certain design choices but also how it works. I found myself having to do double the work on reviewing for my own sake just to understand certain concepts so that I knew what to do on my end. But again, that's the part of learning and I'm glad I took that time to do it. I just want to do it to the highest of levels and standard for next time.

For closing statements GitHub count I don't feel is terribly low. I know and my team does I stayed active and made any contributions they needed me to make. If I made a mistake I would ask them for help. I constantly offered help to my fellow team members as well. Just incredibly thankful to have been grouped with these guys because they ended up becoming some great friends.

Best,
Sebastian E. Cervantez

Re: Team Member Self-Assessment and Contributions

Summarize

DD

Daniel De Jesus <jdejesus1@sfsu.edu>

Today at 10:58 PM

To: Dylan Jon Alec Cordoni; Ryan Zhang; Kenny Camarra; Diego Fernando Rivera-Soto; Abhay Bajaj; Sebastian Emilliano Cervantez

Daniel De Jesus - Front-End Lead

Contributions to the team

- Helped design the prototype of the website and bring it together for the final product
- Applied and adjusted file CSS structure
- Fixed a couple of styling inconsistencies across pages
- Creation of certain pages for our website such as homepage, and create listings
- Helped parts of the milestone documents

Number of submissions

~20 Total commits throughout this project. All though it might be a little low, these commits were usually the stable build I completed for a task.

Main Challenges

I guess the main problem i had while doing this project was that i needed myself to learn more about react and its functionality, since this is my second software dev i had to expose myself to different types of coding environments. Watching tutorials videos and reading articles about react just to reinforce what i know then while simultaneously working in the project was something that took a lot of time from my hand. In the end it all pays out and you have more experience than before.

GenAI

The only thing we used GenAI for are the tasks that we were forced to use it. This is how our name for the website came to be as well as the logo. I think we as a team take pride in what we do to deliver a website using our talents and skills.

In the future

I think the number one thing id do better next time is to never stop practicing. I learned how valuable our time is and what we can accomplish in a given time and i think i couldve done better if i just knew more. Juggling between refreshing my memory about react and doing this project wasn't fun because of the things i could've done more for the website is now lost. However, im grateful that i got to team up with other talented teammates that never showed any resistance or aggression towards one another when something failed during our development

could've done more for the website is now lost. However, im grateful that i got to team up with other talented teammates that never showed any resistance or aggression towards one another when something failed during our development.

Other notes for instructor

I'm not sure if its just me but I feel like future students of 648 should be able to choose their own website to make since it would be cool to see other team doing a completely different website than yours.

Re: Team Member Self-Assessment and Contributions

Summarize

KC

Kenny Camarra <kcamarra@sfsu.edu>

Today at 11:51 PM

To: Dylan Jon Alec Cordoni; Daniel De Jesus; Ryan Zhang; Diego Fernando Rivera-Soto; Abhay Bajaj; Sebastian Emilliano Cervantez

Kenny Camarra - Front-End

Contributions to the Team

- Completing assigned tasks in a timely manner
- Contributed to Milestone Documents & Documentation of Project
- Reviewed pull requests & Contributed to bugfixes
- Assisted in multiple parts of Front-End development of webpages, and Idea creation/implementation
- Designed the original styling/look for the website

Number of Submissions

24 commits to Dev Branch. Grouped together a number of fixes/features into singular commits multiple times; most commits were already tested & integrated builds.

Main Challenges

One of the main challenges throughout this team project was making sure to create good, readable, and easily reusable react components and code. As the person who initially suggested using react, I felt that I needed to step up my game, as initially I only knew the basics of webdev with react from previous courses and minor hobby projects. But being the one who suggested react, I felt the need to learn more, become more proficient, and create components and code that would be easy to work with other group members who have less experience with react. So I made sure to learn more in order to review and bugfix when the need arises; with part of the challenge being coordinating with everyone else on the front-end since we all initially had slightly different ideas on the actual looks of the site before we had a stable, shared idea between the whole team.

GenAI

We did not really use GenAI much as part of our actual finished product, only really using it for minor things like picking a set of names and creating logos for our website. Although we did partially use some GenAI when testing the features when they were incomplete, such as using GenAI to generate mock items that we could use to test search functionality, which worked quite well and saved time from the busywork of handwriting out mock items for testing. We did also initially use GenAI to create part of our vertical prototype, but most of the code and the entire style was redone after Milestone 2, meaning that basically none of it made it to actual production, but it was decently useful to creating a prototype, despite still needing some manual adjustments to make it work according to specs. And on a more personal note, I used GenAI a couple times here and there in order to dissect syntax and logic for react, and it was a great teaching tool.

Next Time

One thing that I could do better next time is time management. Over the course of the project we had some definite downtime that could have been used to schedule more tasks, and do more work so that we would not need to crunch towards the end. I believe we did the best we could with what we knew at the time, but with the benefit of this experience I now know not to let any downtime go to waste if it can save me in the future.

Other Notes for Instructor

I originally didn't feel my github commit count was too low, but do want to bring up that most of my commits were multiple fixes & features all grouped into singular commits that were already integrated into the dev branch, meaning there would be little to no need for cleanup.
All in all, I had a great time with this team and am glad I had them all as my group members as we all grew together over this project and semester, with everyone being reliable and friendly.

Best Regards,
Ken Camarra

Re: Team Member Self-Assessment and Contributions

DF

Diego Fernando Rivera-Soto <driverasoto@sfsu.edu> Today at 11:37 PM

To: Sebastian Emilliano Cervantez; Dylan Jon Alec Cordoni; Daniel De Jesus; Ryan Zhang; Kenny Camarra; Abhay Bajaj

Diego Rivera-Soto - frontend

1. **Contributions to team project:**
   i. Created user personas
   ii. Created use cases
   iii. Worked on story boards and initial page design
   iv. Worded on product summary
   v. Created personal team member page
   vi. Worked on item page
   vii. Created message page
   viii. Created favorites page

b. **Number of submissions they made to GitHub team Dev. Branch:**
The total number of submissions made to GitHub is 17 commits. Although the count may be low, the commits include working on the team member page, item page, message page and favorite page. Most of these commits are saved before commits, fixes to the pages after merging branches and final touchups done at my end.

c. **One brief paragraph on the main challenges they encountered in team project:**
One of the main challenges I encountered while doing the project was that I needed to learn how to use react. My team voted on using React as our main language for our project and since I had no knowledge of this language, I had to learn the basics to be able to understand the structure of our code. In order to do this, I spent time watching YouTube videos of how react worked and how it used HTML, CSS and JavaScript elements. HTML, CSS, and JavaScript are all languages that I already have used before, so after learning the basics of react I was able to figure out how some things worked in order to work on my sections of the project.

d. **One brief paragraph of their own overall experience with GenAI focusing on tasks it was applied to and how useful it was**
Overall, I had a decent experience working with GenAI during the project. One place I used Gen AI was to better understand certain react functions I have never used before. While working on my pages, sometimes I would need to use other components such as the Nav bar. I would post the code from that section and ask the AI to please summarize the code to tell me how the functionality worked and sometimes ask about specific portions of the code. Doing this was pretty helpful since as explained above, I was pretty unfamiliar with react so by doing this I was able to better understand how react worked and understand the code of my fellow peers.

e. **One brief paragraph on what would do better next time based on what was learned in the class about SE management and processes**
Something I would like to do better next time based on what was learned in class about SE management and processes is having more team meetings. Although my team did a great job by mostly meeting during class time, One thing I would like to improve on in future projects is having more meetings with my team to further discuss what we are working on and how it's going. During our project we had a way to send messages to each other and although it was helpful, having team meetings could help more than simple chat messages. Therefore, one thing would do better is try to have more team meetings so that our project can be even better than what it currently is right now

f. **Final thoughts:** Overall had a great time working on the project with my teammates.

Re: Team Member Self-Assessment and Contributions

Summarize

AB

Abhay Bajaj <abajaj@sfsu.edu>

Today at 11:53 PM

To: Kenny Camarra; Dylan Jon Alec Cordoni; Daniel De Jesus; Ryan Zhang; Diego Fernando Rivera-Soto; Sebastian Emilliano Cervantez

Abhay Bajaj - GitHub Organizer/Front-End

Contributions to the Team

- Helped organize and manage the GitHub repository, including branch coordination and keeping development workflow organized
- Contributed to front-end development and UI implementation across multiple pages
- Helped improve styling consistency and responsiveness throughout the website
- Worked with teammates to integrate front-end components with backend functionality
- Participated in milestone discussions, planning, and overall project coordination
- Worked on milestone documents throughout the whole project

Number of Submissions

Around 15 meaningful commits throughout the project. Although the commit count may appear lower compared to some teammates, many of my contributions involved organizing GitHub workflows, helping resolve merge conflicts, reviewing code integration, and assisting teammates with the milestone documents.

Main Challenges

One of the biggest challenges during this project was coordinating development across multiple team members while balancing different schedules and responsibilities outside of class. Since several people were working on related parts of the project at the same time, keeping branches organized and avoiding merge conflicts became increasingly important as the semester progressed. This required constant communication, testing, and adjustments to ensure the website functioned smoothly as a complete product.

GenAI Experience

GenAI was mainly used as a support tool throughout the project. Personally, I found it most useful for troubleshooting coding issues, understanding unfamiliar concepts, and generating ideas when solving problems. As a team, we also used GenAI for brainstorming things such as naming ideas and design inspiration. Overall, it was helpful as a learning and productivity aid, but the majority of the actual implementation and development work was completed by the team ourselves.

What I Would Do Better Next Time

If I could improve anything next time, I would focus on managing pull requests and code integration more efficiently throughout the semester instead of allowing some requests and merges to sit for too long before being reviewed. I learned how important quick communication and organized collaboration are when multiple developers are working on the same project at the same time. Faster pull request reviews and more consistent progress check-ins would have helped reduce merge conflicts and made development smoother during the final stages of the project. In addition, I would continue improving my knowledge of front-end frameworks and collaborative development workflows so I could contribute even more effectively in future team projects.

Other Notes for Instructor

While my GitHub commit count may be lower than some teammates, I remained consistently involved throughout the semester through GitHub organization, troubleshooting, front-end support, and coordination with my whole team. Had a great time working with some great people.

Thank you,
Abhay Bajaj

Re: Team Member Self-Assessment and Contributions

Summarize

RZ

Ryan Zhang <rzhang13@sfsu.edu>

Today at 12:38 PM

To: Abhay Bajaj; Kenny Camarra; Dylan Jon Alec Cordoni; Daniel De Jesus; Diego Fernando Rivera-Soto; Sebastian Emilliano Cervantez

Ryan Zhang - Back-End

Contributions to the Team

- Helped complete and organize Milestone documents
- Created Complete Transaction front-end and back-end
- Implemented Create Listings Page back-end
- Implemented Propose Transaction functionality
- Implemented Delete Listings functionality
- Implemented Admin Approval to creating listings
- Helped resolve merge conflicts and integrate pull requests
- Participated in DB schema
- Debugging

Number of Submissions

At this time ~46 commits. A lot was for debugging and I tried to commit to more frequently in case my VM died as it almost did once.

Main Challenges

One of the biggest challenges was integrating other people's code changes and managing bugs. While generally the areas we worked on didn't overlap too much, when they did we had to figure out what code to keep and what was necessary for the implementation they were pushing. Merging branches also produced a lot of unseen bugs, whether it be inconsistent naming or calling the wrong function, it felt like the more we fixed the bugs the more bugs appeared.

GenAI Experience

GenAI was used a consulting tool as well as a menial task tool. I used it to consult on technical aspects I was learning, like SQL and React, as well as duplicate repetitive code. I also used it to generate test data to input into the database and their corresponding SQL queries. Of course, I had to vet everything it said, with my own knowledge and other sources. I also used it to review code for typos and syntax errors when there was no one else available since I would often be blinded by what the code was supposed to do rather than what was on the screen. It acted as a third eye in this sense. GenAI would

GenAI Experience

GenAI was used a consulting tool as well as a menial task tool. I used it to consult on technical aspects I was learning, like SQL and React, as well as duplicate repetitive code. I also used it to generate test data to input into the database and their corresponding SQL queries. Of course, I had to vet everything it said, with my own knowledge and other sources. I also used it to review code for typos and syntax errors when there was no one else available since I would often be blinded by what the code was supposed to do rather than what was on the screen. It acted as a third eye in this sense. GenAI would especially fail in cases where it didn't know the entirety of our DB schema, so the inserts would be incorrect and I needed to modify it to match. It ultimately did prove a useful tool for accelerating the learning process, but I had to make sure I was probing its output carefully and use my own intuition and sources to ensure correctness.

What I Would Do Better Next Time

If I could do change one thing, it would be to manage my time and time estimates more realistically. I did not anticipate the amount of time that the implementations I was working on would take so long, and as a result I spent every day the last week just coding and debugging and coding again. For example, I would give myself an estimate of a couple hours for a feature, and then I would be 80% done with the feature at the hour and thirty-minute mark and get stuck on a bug for another three hours. If I could do this again, I would spread out the work more evenly and manage my time better, being generous with my time estimates.

Other Notes for Instructor

Like someone else said, I think it would be interesting for students to come up with their own websites so that demo day we see a diverse and unique set of projects, but otherwise the project was fun!

Thank you,
Ryan Zhang